CS 136 — Elementary Algorithm Design and Data Abstraction

Vishesh Gupta

University of Waterloo

Contents

1 Introduction **1**
- 1.1 Course Information . . . 1
  - 1.1.1 Programming environment: Seashell . . . 1
  - 1.1.2 Styled boxes in the course notes . . . 1
  - 1.1.3 Marking scheme . . . 2
- 1.2 What CS 136 Is About . . . 2
- 1.3 Racket Review . . . 3
  - 1.3.1 The design recipe . . . 3
  - 1.3.2 Values, definitions and the substitution model . . . 3
  - 1.3.3 Lists and recursion . . . 3
  - 1.3.4 Structures . . . 4
  - 1.3.5 Higher-order functions . . . 4
  - 1.3.6 Binary search trees and dictionaries . . . 4
- 1.4 Summary . . . 4

2 A Functional Introduction to C **5**
- 2.1 History of C . . . 5
- 2.2 From Racket to C . . . 5
  - 2.2.1 Comments . . . 5
  - 2.2.2 Constants . . . 5
  - 2.2.3 Static typing . . . 6
  - 2.2.4 Initialization . . . 6
- 2.3 Expressions and Operators . . . 6
- 2.4 Functions . . . 6
- 2.5 Hello, World! . . . 7
  - 2.5.1 `printf` placeholders . . . 7
- 2.6 Booleans and Comparisons . . . 7
- 2.7 Conditionals . . . 8
- 2.8 Recursion . . . 8
- 2.9 Declarations vs. Definitions . . . 9
- 2.10 Scope . . . 9
- 2.11 Modules and `#include` . . . 9
- 2.12 Testing with `assert` . . . 9
- 2.13 Floating point . . . 10
- 2.14 Structures . . . 10
- 2.15 Summary . . . 10

3 Modularization and ADTs **11**
3.1 Modularization . . . 11
3.1.1 Terminology . . . 11
3.1.2 Motivation . . . 11
3.2 Modules in Racket . . . 12
3.3 Scope Revisited . . . 12
3.4 Interfaces . . . 13
3.4.1 Example: a sum module in Racket . . . 13
3.4.2 The same module in C . . . 13
3.5 Designing Modules . . . 14
3.5.1 Information hiding . . . 14
3.6 Data Structures and Abstract Data Types . . . 14
3.7 Collection ADTs . . . 15
3.7.1 Dictionary (revisited) . . . 15
3.7.2 Stack . . . 15
3.7.3 Queue . . . 16
3.7.4 Sequence . . . 16
3.8 Summary . . . 16

4 Introduction to Imperative C **17**
4.1 Functional vs. Imperative Programming . . . 17
4.2 Side Effects . . . 17
4.2.1 Statements in C . . . 17
4.2.2 `void` functions . . . 18
4.3 State . . . 18
4.4 Mutation . . . 18
4.4.1 Variables in C . . . 18
4.4.2 Uninitialized variables . . . 19
4.4.3 Compound assignment and increment . . . 19
4.5 Global Mutable State: Why It Is Dangerous . . . 19
4.6 Mutable Structures . . . 20
4.7 Testing Imperative Code . . . 20
4.8 Summary . . . 20

5 The C Model: Memory and Control Flow **21**
5.1 Models of Computation . . . 21
5.2 Memory . . . 21
5.2.1 Bits, bytes and addresses . . . 21
5.2.2 `sizeof` . . . 21
5.2.3 Integer limits and overflow . . . 21
5.2.4 The `char` type and ASCII . . . 22
5.2.5 Structures in memory . . . 22
5.3 Sections of Memory . . . 22
5.3.1 The code section . . . 23
5.3.2 Read-only and global data . . . 23
5.4 Control Flow . . . 23
5.4.1 Return addresses and the call stack . . . 23
5.4.2 Stack frames . . . 24

5.4.3 The stack section . . . 24
5.5 The Model . . . 25
5.6 Control Flow Statements . . . 25
5.6.1 `if` . . . 25
5.6.2 Looping . . . 25
5.6.3 Recursion vs. iteration . . . 27
5.7 Summary . . . 27

6 Pointers **28**
6.1 The Address Operator . . . 28
6.2 Pointers . . . 28
6.2.1 Size of a pointer . . . 28
6.3 The Indirection Operator . . . 29
6.4 The `NULL` Pointer . . . 29
6.5 Pointer Assignment vs. Value Assignment . . . 29
6.6 Mutation Through Parameters . . . 30
6.6.1 The classic example: swap . . . 30
6.6.2 Returning more than one value . . . 31
6.7 Passing Structures . . . 31
6.7.1 `const` pointers . . . 32
6.8 Opaque Structures . . . 32
6.9 Function Pointers . . . 33
6.10 Common Pitfalls . . . 33
6.11 Summary . . . 33

7 I/O and Testing **35**
7.1 Input and Output . . . 35
7.2 Output with `printf` . . . 35
7.3 Debugging Output and Tracing . . . 35
7.4 Input with `scanf` . . . 36
7.4.1 Whitespace . . . 36
7.4.2 Reading all integers until EOF . . . 36
7.4.3 Reading commands . . . 37
7.4.4 Reading characters with `getchar` . . . 37
7.4.5 Invalid input . . . 37
7.5 Testing . . . 38
7.5.1 Why test? . . . 38
7.5.2 Kinds of testing . . . 38
7.5.3 What to test . . . 38
7.5.4 Testing in Seashell . . . 38
7.5.5 Testing modules: a test client . . . 39
7.5.6 Debugging strategies . . . 39
7.6 Summary . . . 39

8 Arrays and Strings **40**
8.1 Arrays . . . 40
8.1.1 Initialization . . . 40
8.1.2 Length and size . . . 41

8.2 Arrays and Pointers . . . 41
8.2.1 Passing arrays to functions . . . 41
8.3 Pointer Arithmetic . . . 41
8.4 Array "Map" . . . 42
8.5 Sorting . . . 42
8.5.1 Selection sort . . . 42
8.5.2 Insertion sort . . . 43
8.5.3 Quicksort . . . 43
8.6 Binary Search . . . 44
8.7 Multi-dimensional Arrays . . . 44
8.8 Fixed-Length Array Implementation of a Stack . . . 44
8.9 Strings . . . 45
8.9.1 String length . . . 46
8.9.2 String I/O . . . 46
8.9.3 `<string.h>` functions . . . 46
8.9.4 String literals . . . 47
8.10 Arrays vs. Pointers . . . 47
8.11 Summary . . . 47

9 Efficiency **48**
9.1 Algorithms and Efficiency . . . 48
9.2 Running Time . . . 48
9.2.1 Worst case . . . 48
9.3 Big-O Notation . . . 48
9.3.1 Rules for simplifying . . . 49
9.3.2 Common orders . . . 49
9.4 Analysing Iterative Code . . . 49
9.5 Analysing Recursive Code . . . 50
9.6 Sorting Algorithms Revisited . . . 51
9.6.1 Insertion sort . . . 51
9.6.2 Quicksort . . . 51
9.6.3 Merge sort . . . 51
9.7 Running Times of Library Functions . . . 52
9.7.1 Racket . . . 52
9.7.2 C . . . 52
9.8 Documenting Efficiency . . . 52
9.9 Space Efficiency . . . 53
9.10 Summary . . . 53

10 Dynamic Memory and ADTs in C **54**
10.1 Motivation . . . 54
10.2 Duration (Lifetime) . . . 54
10.3 `malloc` . . . 54
10.4 `free` . . . 55
10.4.1 Memory errors . . . 55
10.5 Documenting Dynamic Memory . . . 56
10.6 `realloc` . . . 56
10.6.1 Reading a string of unknown length . . . 57

10.7 Dynamic Arrays and Amortized Analysis . . . 57
10.8 ADTs in C . . . 58
10.8.1 Interface: `stack.h` . . . 58
10.8.2 Implementation: `stack.c` . . . 59
10.8.3 Client . . . 60
10.9 Summary . . . 60

11 Linked Data Structures **61**
11.1 Linked Lists . . . 61
11.1.1 Building lists . . . 61
11.1.2 Traversal . . . 61
11.1.3 Destroying a list . . . 62
11.1.4 Mutating lists . . . 62
11.2 A List Wrapper Structure . . . 63
11.2.1 Inserting into a sorted list . . . 63
11.2.2 Removing a node . . . 63
11.3 Augmenting the Wrapper . . . 64
11.3.1 A queue with a linked list . . . 64
11.4 Arrays vs. Linked Lists . . . 65
11.5 Trees . . . 65
11.5.1 BST operations . . . 65
11.5.2 Height and balance . . . 67
11.6 Graphs (briefly) . . . 67
11.7 Summary . . . 67

12 Abstract Data Types in C **68**
12.1 Designing an ADT . . . 68
12.2 The Dictionary ADT . . . 68
12.2.1 Interface . . . 68
12.2.2 Implementation with a BST . . . 69
12.2.3 Removing from a BST . . . 71
12.2.4 Comparing dictionary implementations . . . 72
12.3 Stack, Queue and Sequence Implementations . . . 72
12.3.1 Stack . . . 72
12.3.2 Queue . . . 72
12.3.3 Sequence . . . 73
12.4 Set ADT . . . 73
12.5 Iterating Over an ADT . . . 73
12.6 Summary . . . 74

13 Beyond CS 136 **75**
13.1 Generic Data with `void *` . . . 75
13.1.1 A generic stack . . . 75
13.2 Function Pointers for Generic Algorithms . . . 76
13.2.1 `qsort` . . . 76
13.2.2 A generic BST dictionary . . . 77
13.3 `typedef` . . . 77
13.4 The Compilation Pipeline . . . 78

13.4.1 Include guards 78
13.4.2 Command-line arguments 78
13.5 C vs. Racket vs. Other Languages 79
13.6 Where the Ideas Go Next 79
13.7 Summary 80

Chapter 1

Introduction

1.1 Course Information

Instructors	Tim Brecht and Dan Holtby (my term).
Prerequisites	CS 135, or 85% or higher in CS 115 or CS 116.
Languages	C (the C99 standard) for almost everything, and Racket as a reference point to compare the *functional* paradigm from CS 135 with the *imperative* paradigm of C.
Web page	`http://www.student.cs.uwaterloo.ca/~cs136/`
Textbook	*C Programming: A Modern Approach* (CP:AMA), 2nd ed., by K. N. King. A copy can be borrowed from the DC library on 3-hour reserve.
Course notes	Available on the course web page; a printed coursepack can be bought at Media.Doc (MC 2018).

1.1.1 Programming environment: Seashell

Seashell is a browser-based IDE built at Waterloo specifically for this course.

- It works with both C and Racket.
- It is integrated with the submission system (Marmoset) and the public/private test environment.
- It makes it easy to run your own tests: for a program `foo.c` you can create test input files `foo.in` together with expected output files `foo.expect`.
- You log in with your WatIAM user name and your `student.cs.uwaterloo.ca` password.

1.1.2 Styled boxes in the course notes

The official course notes use three kinds of boxes, and I follow a similar convention:

- **Thick boxes** contain important information (exam material). In these notes they appear as blue "important" boxes.

- **Thin boxes** contain comments and asides. Content in an aside will not be on the exams.
- **Dashed boxes** contain "advanced" material. It may be discussed in class or appear on assignments, but you are not responsible for it on exams unless your instructor says otherwise.

Example of an important box

This is what exam-relevant facts look like in these notes.

And this is what an aside looks like. Asides are not tested.

1.1.3 Marking scheme

Component	Weight
Assignments (roughly weekly)	20%
Participation (clickers)	5%
Midterm	25%
Final exam	50%

Passing the course

To pass CS 136 you must pass *both* the assignment component *and* the weighted exam component (midterm + final).

Clickers. i>Clickers are used for participation marks; they can be bought at the bookstore and are registered as part of Assignment 0. Credit is only given for attending your own section. Using someone else's clicker is an academic offence.

Assignments. Assignments are weekly; depending on the term there are up to 10 (my term had 8).

Advice. Start every assignment early and use the ISA (Instructional Support Assistant) office hours. Write the code on paper first, trace through it by hand, and *then* type it in. Tracing by hand is also exactly what the exams ask you to do.

1.2 What CS 136 Is About

CS 135 taught program design in a purely *functional* setting: values never change and a program is evaluated by *substitution*. CS 136 has three main themes:

1. **Imperative programming.** Programs that work by changing *state* (mutation, side effects, loops). We will need a new model of computation — memory and control flow — to replace the substitution model.
2. **Low-level memory.** C lets (forces) us to deal directly with memory addresses, pointers, the stack and the heap, and to manage memory manually.

3. **Data abstraction and efficiency.** Organising programs into modules, designing abstract data types (stacks, queues, dictionaries, ...), and analysing the efficiency of algorithms and data structures with Big-O notation.

1.3 Racket Review

Since we keep comparing C with Racket, here is a quick review of the CS 135 material we rely on. The course mostly uses the *full* Racket language (`#lang racket`) rather than the teaching languages.

1.3.1 The design recipe

Every function is documented with a *purpose*, a *contract*, and (in Racket) examples/tests:

```
;; (sum-squares lon) produces the sum of the squares of the numbers in lon
;; sum-squares: (listof Num) -> Num
(define (sum-squares lon)
  (cond [(empty? lon) 0]
        [else (+ (sqr (first lon)) (sum-squares (rest lon)))]))

(check-expect (sum-squares '(1 2 3)) 14)
```

In CS 136 we keep the purpose and contract, and add *requires* (preconditions) and later *effects* (side effects) and *time* (efficiency) to the documentation.

1.3.2 Values, definitions and the substitution model

In Racket a `define` binds an identifier to a value that never changes. Evaluation is modelled by repeatedly substituting values for identifiers and applying functions:

```
(define (my-sqr n) (* n n))
(my-sqr (+ 1 2))
=> (my-sqr 3)
=> (* 3 3)
=> 9
```

This works because there are no side effects. Once we introduce mutation, the substitution model breaks down, which is why Unit 05 builds a new model.

1.3.3 Lists and recursion

A list is either `empty` or `(cons v lst)`. Structural recursion follows the shape of the data:

```
;; (count-evens lon) counts the even numbers in lon
;; count-evens: (listof Int) -> Nat
(define (count-evens lon)
  (cond [(empty? lon) 0]
        [(even? (first lon)) (add1 (count-evens (rest lon)))]
        [else (count-evens (rest lon))]))
```

We will also use *accumulative* recursion, which turns into loops in C:

```
(define (sum-acc lon acc)
  (cond [(empty? lon) acc]
        [else (sum-acc (rest lon) (+ acc (first lon)))]))
```

1.3.4 Structures

```
(struct posn (x y))          ; full Racket; define-struct in teaching languages
(define p (posn 3 4))
(posn-x p)                   ; => 3
(posn? p)                    ; => #t
```

C has the same idea with `struct`, but the fields have declared types.

1.3.5 Higher-order functions

Functions are first-class values in Racket: they can be passed as arguments, returned, and created anonymously with `lambda`. The abstract list functions are the classic examples:

```
(map add1 '(1 2 3))                  ; => '(2 3 4)
(filter even? '(1 2 3 4))            ; => '(2 4)
(foldr + 0 '(1 2 3))                 ; => 6
(map (lambda (x) (* x x)) '(1 2 3))  ; => '(1 4 9)
```

C has *function pointers*, which give us a (more limited) version of this (Units 06 and 13).

1.3.6 Binary search trees and dictionaries

A BST node has a key, a value and two subtrees, where every key in the left subtree is smaller and every key in the right subtree is larger than the node's key. A *dictionary* maps keys to values; in CS 135 we implemented it with association lists and BSTs. We revisit both in C in Units 03, 11 and 12.

1.4 Summary

- CS 136 moves from functional Racket to imperative C99, using Seashell.
- Grading: assignments 20%, participation 5%, midterm 25%, final 50%; both the assignment and exam components must be passed.
- The Racket ideas we keep using: the design recipe, the substitution model (and why it fails with mutation), lists and recursion, structures, higher-order functions, BSTs and dictionaries.

Chapter 2

A Functional Introduction to C

Readings: CP:AMA 2.2–2.4, 2.6–2.8, 3.1, 4.1, 5.1, 9.1, 9.2, 10.1, 15.2.

In this unit we write C in a "functional" style: constants only, no mutation, recursion instead of loops. This lets us learn the syntax of C while keeping the CS 135 way of thinking.

2.1 History of C

C was developed by Dennis Ritchie at Bell Labs in 1969–73 to make the Unix operating system more portable. It is the successor of the language B, and C++ is (roughly) its successor. C was designed to give low-level access to memory and to translate easily into machine code, which is why it is still the language of operating systems and embedded systems.

Standard

This course uses the **C99** standard.

2.2 From Racket to C

2.2.1 Comments

```
; single-line comment in Racket
#| multi-line
   comment in Racket |#
```

```
// single-line comment in C (C99)
/* multi-line
   comment in C */
```

2.2.2 Constants

```
(define my-number 42)
```

```
const int my_number = 42;
```

Note: identifiers in C cannot contain `-`, so we use underscores (`my_number`). Every C definition ends in a semicolon.

2.2.3 Static typing

Racket is *dynamically typed*: the type of a value is only known (and checked) at run time, and an identifier can be bound to values of different types:

```
(define dtype (cond [(>= x 0) 42]
                    [else "invalid"]))
```

C is *statically typed*: the type of every identifier must be declared and is known before the program runs. The compiler checks that types are used consistently.

```
const int stype = 42;
```

A consequence is that contracts in C do not need to repeat the types already in the function header; instead we document extra *requirements* (e.g. `n >= 0`).

2.2.4 Initialization

In C, a *definition* reserves storage for an identifier, and *initialization* gives it its first value at the same time. A `const` must be initialized when it is defined, since it can never be assigned afterwards.

2.3 Expressions and Operators

C uses *infix* notation (like ordinary math) instead of Racket's prefix notation: `(+ 1 (* 2 3))` becomes `1 + 2 * 3`. C has precedence rules (`*` binds tighter than `+`); use parentheses when in doubt.

Integer division and remainder

- `/` on integers behaves like Racket's `quotient`: the result is *truncated toward zero* (so `7 / 2` is 3 and `-7 / 2` is -3).
- `%` is the remainder operator: `a % b` has the same sign as `a` (so `-7 % 2` is -1). It is *not* the same as Math 135's mod for negative numbers.
- The identity `(a / b) * b + a % b == a` always holds (for `b != 0`).

2.4 Functions

```
;; (my-sqr n) produces the square of n
;; my-sqr: Int -> Int
(define (my-sqr n)
  (* n n))
```

```
// my_sqr(n) produces the square of n
int my_sqr(int n) {
  return n * n;
}
```

A C function definition gives the return type, the name, the typed parameters, and a body in braces. The `return` statement ends the function and produces the value. A function that takes no parameters is declared with `(void)`, and one that produces nothing has return type `void`.

2.5 Hello, World!

```c
// hello.c: my first C program
#include <stdio.h>

int main(void) {
  printf("Hello, World!\n");
}
```

- Every C program has exactly one `main` function, where execution starts. Its return value is an error code for the operating system: 0 means success. In C99, if `main` reaches the end without a `return`, it returns 0 automatically.
- `#include <stdio.h>` gives access to `printf`.
- `\n` is the newline character.

2.5.1 `printf` placeholders

`printf` takes a *format string* containing placeholders, followed by the values to substitute for them:

```c
printf("%d squared is %d\n", 5, my_sqr(5));   // 5 squared is 25
printf("100%%\n");                             // use %% to print a %
```

Placeholder	Type
`%d`	`int` (decimal)
`%c`	`char`
`%f`	`float`/`double`
`%s`	string (Unit 08)
`%p`	pointer (Unit 06)
`%zd`	`size_t` (Unit 05)

A field width can be given, e.g. `%5d` right-aligns in 5 columns and `%.2f` prints two decimal places.

2.6 Booleans and Comparisons

- In C, *false* is represented by `0` and *true* by `1`. In fact any non-zero value is treated as true in a condition.
- Equality is `==`; inequality is `!=`. Comparisons: `<`, `<=`, `>`, `>=`.
- Logical not `!`, and `&&`, or `||`. Like Racket's `and`/`or`, `&&` and `||` *short-circuit*: the right operand is only evaluated if needed.

`=` vs `==`

`=` is assignment (Unit 04) and `==` is comparison. Writing `if (x = 0)` is a classic bug: it assigns 0 to `x` and the condition is always false.

The `bool` type is not built in; include `<stdbool.h>` to get `bool`, `true` and `false`. Predicate functions should return `bool`:

```c
#include <stdbool.h>

// is_even(n) determines if n is even
bool is_even(int n) {
  return n % 2 == 0;
}
```

2.7 Conditionals

There is no direct C equivalent of `cond`. In functional C we use the *conditional operator* `?:`, which is an *expression*:

```c
int my_abs(int n) {
  return (n >= 0) ? n : -n;
}
```

More generally we use the `if` *statement* (Unit 05 covers it in detail):

```c
int sign(int n) {
  if (n > 0) {
    return 1;
  } else if (n < 0) {
    return -1;
  } else {
    return 0;
  }
}
```

2.8 Recursion

Recursion works exactly the same way as in Racket:

```c
// sum_first(n) produces 0 + 1 + ... + n
// requires: n >= 0
int sum_first(int n) {
  if (n == 0) {
    return 0;
  }
  return n + sum_first(n - 1);
}

// gcd(m, n) produces the greatest common divisor (Euclid)
// requires: m, n >= 0, not both zero
int gcd(int m, int n) {
  if (n == 0) return m;
  return gcd(n, m % n);
}
```

2.9 Declarations vs. Definitions

C reads a file top to bottom; an identifier must be *declared* before it is used. A *declaration* only states the type (for a function: its signature, also called a *prototype*); a *definition* also provides the body/storage.

```
bool is_odd(int n);          // declaration (forward declaration)

bool is_even(int n) {        // definition
  return (n == 0) ? true : is_odd(n - 1);
}

bool is_odd(int n) {
  return (n == 0) ? false : is_even(n - 1);
}
```

Something may be declared many times but defined only once. Declarations are what we put in header (`.h`) files.

2.10 Scope

Identifiers defined inside a function (parameters and local constants) have *local* (block) scope, as in Racket's `local`. Identifiers defined at the top level of a file are *global*. In C, global identifiers have *program* scope by default (visible from other files if declared there); the keyword `static` restricts them to *module* scope (Unit 03).

2.11 Modules and `#include`

`#include` is a *preprocessor directive*: before compilation, the preprocessor textually replaces the line with the contents of the named file.

```
#include <stdio.h>        // standard library module: angle brackets
#include "my_module.h"    // our own module: quotes
```

A module is split into an *interface* file (`.h`: declarations, documentation) and an *implementation* file (`.c`: definitions). This is covered fully in Unit 03.

2.12 Testing with `assert`

`assert(e)` (from `<assert.h>`) does nothing if `e` is true, and stops the program with an error message if `e` is false. We use it to test our functions and to check preconditions ("requires").

```
#include <assert.h>

int main(void) {
  assert(my_sqr(3) == 9);
  assert(sum_first(4) == 10);
  assert(gcd(12, 18) == 6);
}
```

2.13 Floating point

C's `float` and `double` types represent real (non-integer) numbers approximately. They are *inexact*: `0.1 + 0.2 == 0.3` is false. Racket would use exact rationals; C cannot. Avoid floats unless the problem requires them, and never compare floats with `==`.

2.14 Structures

Structures are compound data, like Racket's `struct`, but fields are typed:

```
struct posn {          // structure declaration (note the semicolon)
  int x;
  int y;
};

const struct posn p = {3, 4};          // initialization with braces
const struct posn q = {.y = 4, .x = 3}; // C99 designated initializers

int manhattan(struct posn a, struct posn b) {
  return my_abs(a.x - b.x) + my_abs(a.y - b.y);
}
```

- The type is `struct posn` (the word `struct` is part of it).
- Fields are accessed with the *selection operator* `.` (e.g. `p.x`).
- `==` does *not* work on structures; compare field by field.
- Unlike Racket, there is no automatic constructor/predicate; a structure is a block of memory holding its fields.

2.15 Summary

- C99 is statically typed; definitions end with `;`; functions declare return and parameter types.
- `/` truncates toward zero; `%` is the remainder with the sign of the dividend.
- 0 is false, non-zero is true; use `<stdbool.h>` for `bool`.
- `?:` is the conditional expression; `if` is a statement.
- Declarations vs. definitions; `#include` is textual inclusion by the preprocessor.
- Test with `assert`; use `struct` for compound data and `.` to select fields.

Chapter 3

Modularization and ADTs

Readings: CP:AMA 15 (writing large programs), 19.1–19.3.

3.1 Modularization

Definition 3.1. *Modularization* is dividing a program into smaller, well-defined pieces called *modules*. A module provides a collection of functions (and possibly other things, such as constants or data structures) that share a common aspect or purpose.

We have been doing a small version of this since CS 135 with helper functions. A module is the next level up: a whole group of related functions.

Modules vs. files. It is good style to store each module in its own file (in C, two files: `.h` and `.c`). In these notes "module" and "file" are used almost interchangeably.

3.1.1 Terminology

- A *client* is a module (or program) that requires a function a module provides.
- A module may itself be a client of other modules. The "requires" relationships form a *module dependency graph*.
- The dependency graph must **not have cycles** (module A cannot require B if B requires A).
- There must be a *root* (main) file that is only a client and is not required by anything; in C it is the file that contains `main`.

3.1.2 Motivation

1. **Re-usability.** A well-written module (e.g. a dictionary) can be used by many programs.
2. **Maintainability.** Bugs can be found and fixed in one place; modules can be tested independently; big projects can be split among team members.
3. **Abstraction.** A client only needs to know *what* a module does, not *how* it does it.

In computer science, modularization is also known as the *separation of concerns* (SoC).

3.2 Modules in Racket

```
;; fun.rkt
(provide fun?)

(define lofn '(-3 7 42 136 1337 4010 8675309))

;; (fun? n) determines if n is a fun integer
;; fun?: Int -> Bool
(define (fun? n)
  (not (false? (member n lofn))))
```

The `provide` special form makes `fun?` available to clients; `lofn` is not provided, so it is private to the module.

```
;; client.rkt
(require "fun.rkt")
(fun? 4010)    ; => #t
(fun? 4011)    ; => #f
```

3.3 Scope Revisited

- **Local scope:** the identifier is only visible inside the local region (e.g. the function body or block) where it is defined.
- **Global scope:** the identifier is defined at the top level and is visible to all code that follows its definition.

Modules split global scope into two levels:

- **Module scope:** visible only inside the module it is defined in.
- **Program scope:** visible outside the module (to clients) as well.

In Racket, identifiers are module scope unless `provided`. In C it is the other way round.

Scope in C

Global identifiers in C have *program* scope by default. Adding the keyword `static` to a global definition gives it *module* scope:

```
static const int secret = 42;      // module scope
static int helper(int n) { ... }   // module scope ("private" helper)
int visible(int n) { ... }         // program scope
```

(The keyword `static` means something different on local variables; we avoid that use.)

To use a program-scope identifier from another file, the client must *declare* it. For variables this needs the keyword `extern` (`extern int counter;`); for functions a prototype is enough. Of course clients should get these declarations by including the module's header file rather than writing them by hand.

3.4 Interfaces

Definition 3.2. A module's *interface* is the list of functions (and other identifiers) it provides, together with their documentation. The *implementation* is the code that defines them.

Interface documentation should contain:

- an overall description of the module;
- the list of functions it provides;
- the contract and purpose (and requires/effects/time) of each provided function.

The interface should *not* describe how things are implemented; that is the point.

3.4.1 Example: a sum module in Racket

```
;; sum.rkt: a module for summing numbers
(provide sum-first sum-squares)

;; (sum-first n) produces 1 + 2 + ... + n
;; sum-first: Nat -> Nat
;; (sum-squares n) produces 1^2 + 2^2 + ... + n^2
;; sum-squares: Nat -> Nat

;;;;;;;;;;;;;;;; IMPLEMENTATION ;;;;;;;;;;;;;;;;
(define (sum-first n)
  (/ (* n (+ n 1)) 2))

(define (sum-squares n)
  (/ (* n (+ n 1) (+ (* 2 n) 1)) 6))

(define (private-helper p) ...)   ; not provided
```

3.4.2 The same module in C

```
// sum.h -- INTERFACE
// a module for summing numbers

// sum_first(n) produces 1 + 2 + ... + n
// requires: n >= 0
int sum_first(int n);

// sum_squares(n) produces 1^2 + 2^2 + ... + n^2
// requires: n >= 0
int sum_squares(int n);
```

```
// sum.c -- IMPLEMENTATION
#include "sum.h"

int sum_first(int n) {
  return n * (n + 1) / 2;
```

```c
}

int sum_squares(int n) {
  return n * (n + 1) * (2 * n + 1) / 6;
}
```

```c
// main.c -- CLIENT (and test client)
#include <assert.h>
#include "sum.h"

int main(void) {
  assert(sum_first(1) == 1);
  assert(sum_first(10) == 55);
  assert(sum_squares(3) == 14);
}
```

The implementation includes its own header so the compiler can check that the declarations and definitions match.

Testing. It is good practice to write a separate *testing client* for each module that calls every provided function, so that all tests should pass (in Racket every test expression should produce `#t`; in C every `assert` should succeed).

3.5 Designing Modules

Goal: high cohesion, low coupling

- **High cohesion:** all of the interface functions of a module are related and work towards a common goal.
- **Low coupling:** there is little interaction between modules. It is impossible to remove all coupling, but it should be minimal.

3.5.1 Information hiding

A module should hide its implementation details from clients:

- **Security:** we may want to prevent the client from tampering with the data used by the module (and breaking its invariants).
- **Flexibility:** if clients cannot depend on implementation details, we are free to change the implementation later (e.g. for efficiency) without breaking any client.

3.6 Data Structures and Abstract Data Types

Consider a module for storing a single account (an ID and a balance). We could implement it with three different data structures: a two-element list, a structure with two fields, or a nested structure. If clients only use the interface functions (`account-id`, `account-balance`, ...), they cannot tell which one is used.

Definition 3.3. An *abstract data type* (ADT) is a mathematical model for storing and accessing data through a set of operations. It is described only by the behaviour of its operations, not by how the data is stored.

Data structure vs. ADT

- With a **data structure**, you know how the data is organised and you can access it directly in any way you like (e.g. a Racket list, a C array).
- With an **ADT**, you do not know how the data is stored; you can only access it through the interface functions (operations) the ADT provides.

An ADT is *implemented* using some data structure, and the same ADT may have several implementations with different efficiency.

In Racket it is hard to truly hide a structure from clients. In C we will use *opaque structures* (Units 06 and 10) to enforce the abstraction.

3.7 Collection ADTs

A *collection ADT* stores an arbitrary number of items and has well-defined operations for adding, removing and accessing them.

3.7.1 Dictionary (revisited)

The *dictionary* ADT (also called a map, associative array, or symbol table) is a collection of *key/-value pairs* with unique keys. (JSON objects in JavaScript are dictionaries.) Typical operations:

- `lookup`: given a key, produce the corresponding value (or "not found");
- `insert`: add a new key/value pair (or replace the value of an existing key);
- `remove`: delete the pair with a given key.

A dictionary can be implemented with an association list, a (sorted) array, or a binary search tree (as in CS 135 — we implement it in C in Unit 12).

3.7.2 Stack

A *stack* is a collection where items are "stacked" on top of each other. Items are *pushed* onto the top and *popped* off the top, so the last item in is the first item out: LIFO. Think of a stack of plates. Uses: browser "back" history, undo in text editors, the call stack (Unit 05), matching brackets.

Operations:

- `push`: add an item to the top;
- `pop`: remove the top item;
- `top`: produce the top item (without removing it);
- `is_empty`: determine if the stack is empty.

Stack vs. Racket list. The operations correspond closely to `cons`, `rest`, `first` and `empty?`. The significant difference is that only the top item of a stack is accessible, whereas any element of a list can be accessed (e.g. with `list-ref`).

3.7.3 Queue

A *queue* is a collection where items are added at the back and removed from the front: first in, first out (FIFO), like a line-up at a restaurant. Operations:

- `add_back`: add an item to the back;
- `remove_front`: remove the front item;
- `front`: produce the front item;
- `is_empty`: determine if the queue is empty.

3.7.4 Sequence

A *sequence* ADT is useful when you want to retrieve, insert or delete at *any* position. Operations:

- `item_at`: produce the item at a given position;
- `insert_at`: insert an item at a given position (later items shift back);
- `remove_at`: remove the item at a given position;
- `length`: the number of items.

> Many languages' "list" types (Python's `list`, C++'s `std::vector`) are really sequence ADTs. Whether `item_at` is fast depends on the implementation: $\mathcal{O}(1)$ with an array, $\mathcal{O}(n)$ with a linked list (Unit 09).

3.8 Summary

- Modules group related functions; the dependency graph is acyclic with a single root.
- Interfaces (`.h`) contain declarations and documentation; implementations (`.c`) contain definitions.
- C globals have program scope by default; `static` makes them module scope.
- Aim for high cohesion and low coupling; hide information for security and flexibility.
- An ADT is defined by its operations, a data structure by its layout.
- Collection ADTs: dictionary, stack (LIFO), queue (FIFO), sequence.

Chapter 4

Introduction to Imperative C

Readings: CP:AMA 4.2–4.5, 5.2, 7.1, 10.2–10.4.

4.1 Functional vs. Imperative Programming

Definition 4.1. In the *functional* paradigm, programs only use constant values that never change; functions produce values and have no other effect. In the *imperative* paradigm, a program is a sequence of *statements* (commands) that change the *state* of the program.

Racket is mostly functional (but has imperative features); C is primarily imperative.

4.2 Side Effects

Definition 4.2. An expression or function has a *side effect* if it does more than produce a value: it also changes the state of the program (or "the world").

Printing is the simplest side effect: `printf` changes what is on the screen. In Racket, `printf` produces `(void)`, which is useless as a value; we only call it for its effect. To do several things in sequence in Racket we use `begin`, which evaluates each expression in order and produces the value of the *last* one (the values of the others are ignored):

```
(define (noisy-sqr n)
  (begin
    (printf "squaring ~a\n" n)
    (* n n)))
```

Function bodies and `cond` answers in full Racket have an implicit `begin`.

4.2.1 Statements in C

C functions are written as sequences of statements:

- an *expression statement* is an expression followed by a semicolon; its value is discarded, so it is only useful for its side effects (e.g. `printf("hi\n");`);
- a *compound statement* (block) is a sequence of statements in braces `{ ... }`;
- *control flow statements* (`return`, `if`, loops) change the order of execution (Unit 05).

4.2.2 `void` functions

A function that only has side effects and produces nothing has return type `void`. It may use `return;` (no value) to exit early, or simply reach the end of its body.

```c
// print_square(n) prints the square of n
// effects: produces output
void print_square(int n) {
  printf("%d squared is %d\n", n, n * n);
}
```

Documenting side effects

If a function has a side effect, its documentation must contain an **effects:** section, e.g. "effects: produces output" or "effects: modifies *p" or "effects: modifies the global counter".

4.3 State

The biggest difference between the imperative and functional paradigms is the existence of side effects that change the state.

Definition 4.3. The *state* of a program is the value of all of its data at a moment in time.

In a functional program, the result of a function call depends only on its arguments. In an imperative program it may also depend on the state, so the same call can give different results at different times.

4.4 Mutation

Definition 4.4. *Mutation* is changing the value of a variable (or of data in memory).

In Racket, `set!` mutates a variable:

```racket
(define count 0)
(define (increment!)
  (set! count (add1 count))
  count)
(increment!)  ; => 1
(increment!)  ; => 2   (same expression, different value!)
```

4.4.1 Variables in C

A definition *without* `const` creates a mutable *variable*:

```c
int count = 0;           // mutable global variable
const int limit = 10;    // constant: cannot be changed

int increment(void) {
  count = count + 1;     // assignment
  return count;
}
```

Assignment

`x = e` is the *assignment operator*. It evaluates `e`, stores the result in `x`, and (as an expression) also produces that value. Initialization (`int x = 5;`) and assignment (`x = 5;`) look similar but are different: `const` variables may be initialized but never assigned.

Because assignment is an expression, `x = y = 0;` works (assigns right to left), and `if (x = 0)` compiles. The latter is almost always a bug. Some people write `if (0 == x)` so that the typo `0 = x` gives a compile error.

4.4.2 Uninitialized variables

In C, a variable can be defined without an initial value: `int x;`. A *global* uninitialized variable is automatically set to 0, but a *local* one contains whatever arbitrary bits were left in that memory. Reading it is a bug. Always initialize variables.

4.4.3 Compound assignment and increment

Shorthand	Meaning
`x += y`	`x = x + y`
`x -= y`	`x = x - y`
`x *= y`, `x /= y`, `x %= y`	similarly
`++x`	increment x, produce the *new* value
`x++`	increment x, produce the *old* value
`--x`, `x--`	likewise for decrement

```
int x = 5;
int a = x++;   // a is 5, x is 6
int b = ++x;   // b is 7, x is 7
```

Prefer `++x` when the value is not used. Never use `x++` twice on the same variable in one expression (e.g. `x = x++ + x++;`): that is undefined behaviour.

4.5 Global Mutable State: Why It Is Dangerous

Mutable global variables make programs harder to reason about and test, because any function anywhere could change them. They also increase coupling between modules. In CS 136 they are allowed but discouraged; most state should be passed to functions explicitly (with pointers, Unit 06). When a module does need private state, make the variable `static` so only that module can change it:

```
// counter.c
static int count = 0;      // module-scope state, hidden from clients

// increment() increases the counter by one and produces the new value
// effects: modifies the module's counter
int increment(void) {
  ++count;
```

```
  return count;
}

void reset(void) {
  count = 0;
}
```

4.6 Mutable Structures

Fields of a non-`const` structure can be mutated with assignment:

```
struct posn p = {1, 2};
p.x = 5;           // p is now {5, 2}
p.y += p.x;        // p is now {5, 7}

struct posn q = p; // copies all fields; q is independent of p
q.x = 0;           // p.x is still 5
```

Assigning one structure to another copies every field. This matters a lot once we pass structures to functions (Units 05 and 06).

4.7 Testing Imperative Code

With side effects, testing must check both the produced value and the changed state:

```
int main(void) {
  reset();
  assert(increment() == 1);
  assert(increment() == 2);
  reset();
  assert(increment() == 1);
}
```

Testing output requires comparing the printed output against an expected output file (Unit 07).

4.8 Summary

- Imperative programs are sequences of statements that change state.
- A side effect is any change of state (output, mutation); document it under *effects*.
- Variables without `const` are mutable; `=` assigns; `+=`, `++x`, `x++` are shorthands.
- Always initialize local variables; avoid mutable globals (or make them `static`).
- Structure assignment copies every field.

Chapter 5

The C Model: Memory and Control Flow

Readings: CP:AMA 5.2–5.3, 6, 7.1–7.3, 9.2–9.3, 18.2.

5.1 Models of Computation

In CS 135 we modelled Racket's behaviour with *substitution*. Substitution cannot handle mutation, so in CS 136 we model C with two components:

- **memory**: what data is stored where, and
- **control flow**: which statement is being executed, and what runs next.

5.2 Memory

5.2.1 Bits, bytes and addresses

A *bit* of storage has two possible states: 0 or 1. A *byte* is 8 bits, so a byte has $2^8 = 256$ possible states. Memory is a huge array of bytes; the position of a byte is its *address*. To access data we go to its address. Addresses are usually written in hexadecimal, e.g. `0x5000`.

5.2.2 `sizeof`

When C defines a variable it reserves enough consecutive bytes for its type. The `sizeof` operator (it looks like a function but is an operator evaluated at compile time) gives the number of bytes a type or expression uses. Its result has type `size_t`, printed with `%zd`.

```
printf("%zd\n", sizeof(int));    // 4 (in Seashell)
printf("%zd\n", sizeof(char));   // 1 (always)
int n = 0;
printf("%zd\n", sizeof n);       // 4
```

5.2.3 Integer limits and overflow

An `int` in Seashell is 4 bytes = 32 bits, so there are only 2^{32} possible values. Integers are *signed*: the range is -2^{31} to $2^{31} - 1$. The limits are provided by `<limits.h>`:

```c
#include <limits.h>
INT_MAX   // 2147483647  =  2^31 - 1
INT_MIN   // -2147483648  = -2^31
```

(You do not need to memorise these values.)

Overflow

If an arithmetic result is outside the range of its type, *overflow* occurs. For signed `int`, overflow is **undefined behaviour**: anything may happen (in practice the value usually "wraps around", e.g. `INT_MAX + 1` becomes `INT_MIN`). It is your job to make sure overflow cannot happen.

`unsigned int` uses the same 32 bits to store 0 to $2^{32} - 1$, and overflow of unsigned values is well defined (arithmetic modulo 2^{32}). `long long` is at least 64 bits.

5.2.4 The `char` type and ASCII

A `char` is 1 byte and is used to store a single character. It is really a small integer: characters are encoded using ASCII (American Standard Code for Information Interchange), which assigns codes 0–127, e.g. `'A'` is 65, `'a'` is 97, `'0'` is 48, `' '` is 32 and `'\n'` is 10.

```c
char c = 'A';       // the same value as
char d = 65;        // ... this
printf("%c %d\n", c, c);         // A 65
char lower = c + ('a' - 'A');    // 'a'
int digit = '7' - '0';           // 7
```

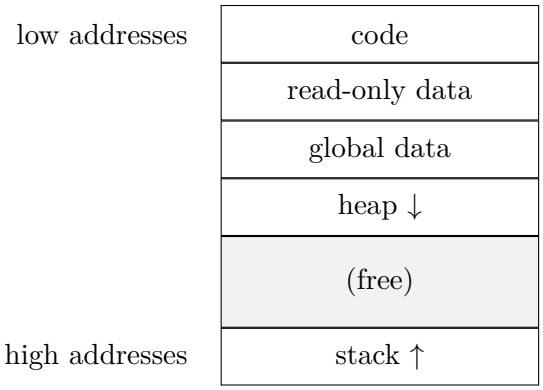
The placeholder for `char` is `%c`. Single quotes give a character; double quotes give a string (Unit 08). Many assignment problems are solved by doing arithmetic on characters as above.

5.2.5 Structures in memory

A structure occupies (at least) the sum of the sizes of its fields; the compiler may add *padding* between fields for alignment, so `sizeof(struct s)` can be larger than the sum. Always use `sizeof` rather than computing sizes by hand.

5.3 Sections of Memory

The memory available to a C program is divided into five *sections* (also called segments):



The operating system and hardware enforce the boundaries: trying to access memory outside the sections your program owns (or writing to a read-only section) causes a **segmentation fault**, and the program crashes.

5.3.1 The code section

Source code is the human-readable ASCII text we write. The compiler translates it into *machine code*, which the processor can execute; this translation is called *compiling*. When the program runs, the machine code is placed in the code section. (CS 241 shows how this is done.)

5.3.2 Read-only and global data

Global (top-level) identifiers are stored in one of these two sections:

- global *constants* go in the **read-only data** section;
- global *variables* go in the **global data** section.

These are created and initialized *before* `main` runs, and exist until the program ends. This is why global initializers must be constant expressions.

The heap is covered in Unit 10, and the stack below.

5.4 Control Flow

5.4.1 Return addresses and the call stack

When a function is called, the program "jumps" to the function's code. When it finishes, execution continues where the call was made. So for each call we must remember the *return address*: the location in the caller to go back to.

Remark. The return address is not the same as the `return` statement. `return` produces a value for the caller; the return address says *where* in the caller to resume. Even `main` has a return address (back into the operating system).

Calls are nested: `main` calls `f`, which calls `g`, ..., and they return in the reverse order. So the history of calls is a *stack* (LIFO), called the *call stack*. It is the same idea as the stack ADT: a call pushes, a return pops.

5.4.2 Stack frames

Each function call pushes a *stack frame* (activation record) containing:

- the argument values (parameters),
- the local variables,
- the return address.

Each call gets a new frame, whether it is a call to a different function or a recursive call to the same function.

Pass by value

C makes a **copy** of each argument value and stores the copy in the new stack frame. This is called *pass by value*. Changing a parameter inside a function does not change the caller's variable. (Unit 06 shows how to emulate *pass by reference* with pointers.)

Example 5.1.

```c
int fact(int n) {
  if (n <= 1) return 1;
  return n * fact(n - 1);   // fact:7
}

int main(void) {
  int r = fact(3);          // main:11
  printf("%d\n", r);
}
```

When `fact(1)` is executing, the call stack looks like this (top of the stack drawn at the top):

fact n: 1 return address: fact:7
fact n: 2 return address: fact:7
fact n: 3 return address: main:11
main r: ??? return address: OS

5.4.3 The stack section

The call stack is stored in the stack section of memory. Each frame uses memory, so very deep recursion (or very large local variables) can exhaust it: a **stack overflow**, which usually crashes the program with a segmentation fault. The stack section in Seashell is relatively small (a few megabytes), so recursion depth of around a million is already a problem.

5.5 The Model

At any moment, the state of a running C program is:

1. the *current program location* (which statement is being executed), and
2. the current contents of memory.

Calling a function.

1. A stack frame is created and pushed; argument values are copied into it.
2. The current program location is stored in the frame as the return address.
3. The program location jumps to the start of the function.

Local variables are added to the frame as their definitions are reached.

Returning.

1. The return value (if any) is noted.
2. The program location is changed back to the frame's return address.
3. The stack frame is popped (its memory is no longer valid).

5.6 Control Flow Statements

5.6.1 `if`

```
if (condition) {
  // block
}

if (condition) {
  // block
} else {
  // block
}

if (c1) {
  // block
} else if (c2) {
  // block
} else {
  // block
}
```

Braces may be omitted for a single statement, but that is error-prone ("dangling else"); CS 136 style always uses braces.

5.6.2 Looping

With mutation we can repeat a block until some state changes, which replaces most uses of recursion.

while.

```
// sum_first(n) produces 1 + ... + n
int sum_first(int n) {
  int sum = 0;
  int i = 1;
  while (i <= n) {
    sum += i;
    ++i;
  }
  return sum;
}
```

The condition is checked before each iteration; if it is false initially the body never runs. A common bug is forgetting to update the loop variable, giving an infinite loop.

do ... while. The body runs at least once, and the condition is checked after each iteration:

```
do {
  // body
} while (condition);
```

for. The most common loop combines initialization, condition and update:

```
for (setup; condition; update) {
  body;
}
// is equivalent to
setup;
while (condition) {
  body;
  update;
}
```

```
int sum = 0;
for (int i = 1; i <= n; ++i) {   // i is local to the loop (C99)
  sum += i;
}
```

Nested loops.

```
// print_triangle(n) prints a right triangle of stars with n rows
// effects: produces output
void print_triangle(int n) {
  for (int row = 1; row <= n; ++row) {
    for (int col = 0; col < row; ++col) {
      printf("*");
    }
    printf("\n");
  }
}
```

`break` and `continue`. `break` exits the innermost loop immediately; `continue` skips the rest of the current iteration and goes to the next one (in a `for` loop, the update still runs).

```
// is_prime(n) determines if n is prime
bool is_prime(int n) {
  if (n < 2) return false;
  for (int d = 2; d * d <= n; ++d) {
    if (n % d == 0) {
      return false;          // return also exits the loop
    }
  }
  return true;
}
```

5.6.3 Recursion vs. iteration

Accumulative recursion translates directly into a loop: the accumulators become variables that are mutated each iteration. A loop uses a single stack frame, so it cannot overflow the stack.

5.7 Summary

- Memory is an array of bytes with addresses; `sizeof` gives sizes in bytes.
- `int` has a limited range (`INT_MIN` to `INT_MAX`); signed overflow is undefined behaviour. `char` is a 1-byte integer holding an ASCII code.
- Five sections: code, read-only data, global data, heap, stack. Bad access causes a segmentation fault.
- Each call pushes a stack frame (arguments, locals, return address); arguments are passed by value. Too many frames cause a stack overflow.
- `if`, `while`, `do`–`while`, `for`, `break`, `continue`.

Chapter 6

Pointers

Readings: CP:AMA 11, 17.7.

6.1 The Address Operator

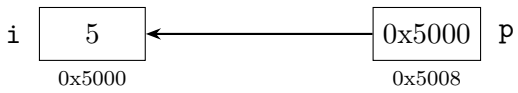
The *address operator* `&` produces the (starting) address of where the value of an identifier is stored in memory. The `printf` placeholder for an address is `%p` (printed in hexadecimal).

```
int g = 42;
int main(void) {
  printf("the value of g is %d\n", g);
  printf("the address of g is %p\n", &g);   // e.g. 0x601040
}
```

6.2 Pointers

Definition 6.1. A *pointer* is a value that is an address. A pointer variable is defined by placing a star `*` before the identifier.

```
int i = 5;
int *p = &i;    // p "points at" i
```



The type of `p` is "pointer to int", written `int *`. Pointers have types: an `int *` should only point at `int`s, a `char *` at `char`s, and so on.

> The placement of the space does not matter: `int *p`, `int* p` and `int * p` are all the same. CS 136 style puts the star next to the identifier because `int *p, q;` defines a pointer `p` and an *int* `q`. Avoid defining several pointers on one line.

6.2.1 Size of a pointer

On a k-bit system, addresses (and hence all pointers, regardless of the type they point to) are k bits. Seashell is a 64-bit system, so `sizeof(int *)` is **8 bytes**.

6.3 The Indirection Operator

The *indirection* operator `*` (also called the *dereference* operator) is the inverse of `&`: `*p` is "the value at the address `p`".

```
int i = 5;
int *p = &i;
printf("%d\n", *p);   // 5
*p = 7;               // changes i!
printf("%d\n", i);    // 7
```

`*&i` is the same as `i`. Note that the star means two different things: in a definition (`int *p`) it is part of the type, and in an expression (`*p`) it is the indirection operator.

Pointers can point to pointers:

```
int i = 5;
int *p = &i;
int **pp = &p;     // pointer to pointer to int
**pp = 6;          // i is now 6
```

6.4 The `NULL` Pointer

`NULL` (defined in `<stdlib.h>` and `<stdio.h>`, among others) is a special pointer value that points to nothing; it is address 0. It is used to represent "no value" (like `empty` or `#f` in Racket).

Dereferencing `NULL`

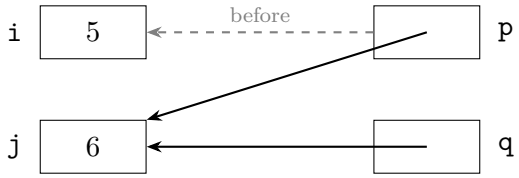
Dereferencing a `NULL` pointer (or any invalid pointer) is undefined behaviour; in practice the program crashes with a segmentation fault. Since `NULL` is false and every other pointer is true, `assert(p);` asserts that `p` is not `NULL`. Functions that take pointers should usually start with such asserts (and the documentation should say "requires: p is a valid pointer").

6.5 Pointer Assignment vs. Value Assignment

This is the single most important idea in the unit.

```
int i = 5;
int j = 6;
int *p = &i;
int *q = &j;
p = q;        // pointer assignment
```

`p = q` changes *what `p` points at*: now both `p` and `q` point at `j`, so `*p` is 6. `i` is still 5.



```c
int i = 5;
int j = 6;
int *p = &i;
int *q = &j;
*p = *q;      // value assignment
```

`*p = *q` copies the *value* that `q` points at into the location `p` points at. `p` still points at `i`, but now `i` is 6.

Definition 6.2. When two (or more) pointers point to the same location, or a pointer points to a named variable, we say they are *aliases*: the same memory can be changed through either name. After `p = q` above, `*p` and `*q` are aliases for `j`.

Aliasing makes code harder to reason about: changing `*p` silently changes `*q`.

6.6 Mutation Through Parameters

Since C passes arguments by value, a function cannot change the caller's variables directly. But if we pass the *address* of a variable, the function can change the variable through the pointer. This emulates *pass by reference*.

```c
// inc(p) increments the integer p points at
// requires: p is a valid pointer
// effects: modifies *p
void inc(int *p) {
  assert(p);
  *p += 1;
}

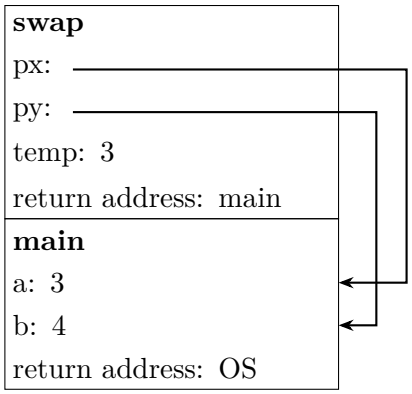
int main(void) {
  int x = 5;
  inc(&x);              // x is now 6
}
```

6.6.1 The classic example: swap

```c
// swap(px, py) swaps the values of *px and *py
// effects: modifies *px and *py
void swap(int *px, int *py) {
  int temp = *px;
  *px = *py;
  *py = temp;
}

int a = 3, b = 4;
swap(&a, &b);    // a is 4, b is 3
```

A version `void swap(int x, int y)` would only swap the copies in its own stack frame and have no effect on the caller.



6.6.2 Returning more than one value

A function can only `return` one value, but it can "produce" more by storing results through pointer parameters (*output parameters*):

```
// divide(num, denom, quot, rem) stores num / denom in *quot
//   and num % denom in *rem
// requires: denom != 0, quot and rem are valid pointers
// effects: modifies *quot and *rem
void divide(int num, int denom, int *quot, int *rem) {
  assert(denom != 0);
  assert(quot && rem);
  *quot = num / denom;
  *rem = num % denom;
}
```

A common pattern is to return a `bool` success flag and store the actual result through a pointer (this is exactly how `scanf` works, Unit 07).

Never return the address of a local variable

```
int *bad(void) {
  int x = 5;
  return &x;      // x's frame is popped on return: dangling pointer!
}
```

Once the function returns, its stack frame is gone; the pointer now points at memory that will be reused by the next call. Using it is undefined behaviour.

6.7 Passing Structures

Passing a structure by value copies the entire structure into the new stack frame. For large structures that wastes time and stack space (and can contribute to stack overflow). Instead we pass a pointer, which is always 8 bytes in Seashell regardless of the structure size.

Fields are accessed through a pointer with `(*ptr).field`. The parentheses are needed because `.` has higher precedence than `*`. This is so common that C provides the *arrow operator*, which combines indirection and selection: `ptr->field`.

```
struct posn { int x; int y; };
```

```c
// posn_scale(p, k) multiplies both coordinates of *p by k
// effects: modifies *p
void posn_scale(struct posn *p, int k) {
  assert(p);
  p->x *= k;           // same as (*p).x *= k
  p->y *= k;
}
```

6.7.1 `const` pointers

If a function takes a pointer only for efficiency and does not modify the data, mark it `const` so the compiler enforces (and documents) that:

```c
int posn_dist_sq(const struct posn *a, const struct posn *b) {
  int dx = a->x - b->x;
  int dy = a->y - b->y;
  return dx * dx + dy * dy;
}
```

The position of `const` matters:

```c
const int *p;          // p can change; *p cannot be changed through p
int * const q = &i;    // q cannot change; *q can
const int * const r = &i;  // neither can change
```

(Read declarations right to left: "`q` is a const pointer to an int".)

6.8 Opaque Structures

C allows a structure to be *declared* without being *defined*:

```c
struct account;     // incomplete type: fields unknown
```

A client that only sees this declaration cannot create a `struct account` or access its fields (it does not know its size or fields), but it *can* use pointers to it (`struct account *`), since all pointers have the same size. This is an *opaque structure*, and it is how C enforces information hiding for ADTs: the header declares the structure, the implementation file defines it, and clients can only use the interface functions.

```c
// account.h
struct account;
struct account *account_create(int id, int balance);   // Unit 10
int account_balance(const struct account *acc);
void account_deposit(struct account *acc, int amount);
```

```c
// account.c
#include "account.h"
struct account {       // the complete definition is private to this file
  int id;
  int balance;
};
int account_balance(const struct account *acc) {
```

```
  return acc->balance;
}
```

Creating opaque structures requires dynamic memory, which we see in Unit 10.

6.9 Function Pointers

In Racket, functions are first-class values. In C, functions are not values, but *pointers to functions* are: the name of a function evaluates to the address of its code (in the code section). This lets us pass a function to another function.

```
int my_sqr(int n) { return n * n; }
int my_neg(int n) { return -n; }

// a function pointer variable: f points to a function int -> int
int (*f)(int) = my_sqr;
printf("%d\n", f(5));     // 25
f = my_neg;
printf("%d\n", f(5));     // -5

// apply_twice(f, x) produces f(f(x))
int apply_twice(int (*f)(int), int x) {
  return f(f(x));
}
apply_twice(my_sqr, 3);   // 81
```

The syntax `int (*f)(int)` reads: "`f` is a pointer to a function taking an `int` and returning an `int`". The parentheses around `*f` are needed: `int *f(int)` would declare a function returning an `int *`. We use function pointers for "map"-style array functions (Unit 08) and generic ADTs (Unit 13).

6.10 Common Pitfalls

- Dereferencing an uninitialized or `NULL` pointer (segmentation fault or worse).
- Returning the address of a local variable (dangling pointer).
- Confusing `p = q` (pointer assignment) with `*p = *q` (value assignment).
- Writing `*p.x` instead of `p->x`.
- Forgetting `&` when passing a variable that should be modified (e.g. `scanf("%d", n)`).

6.11 Summary

- `&x` is the address of `x`; `*p` is the value at address `p`.
- Pointers are typed; all pointers are 8 bytes in Seashell; `NULL` points nowhere.
- Pass a pointer to let a function modify the caller's data (emulated pass by reference); use output parameters to produce several results.

- Pass structures by pointer; use `->`; use `const` for read-only access.
- Opaque structures (declared but not defined in the header) enforce ADT abstraction.
- Function pointers let functions be passed as arguments.

Chapter 7

I/O and Testing

Readings: CP:AMA 2.5, 3.1–3.2, 22.1–22.3 (selected).

7.1 Input and Output

Definition 7.1. *Input and output* (I/O) is how a program interacts with the outside world: receiving data (input) and displaying or sending data (output).

A program may interact with humans (keyboard, screen) or with non-human entities: files, other programs, or other computers over a network (e.g. a web server). In CS 136 we only use *standard input* (stdin, usually the keyboard) and *standard output* (stdout, usually the screen). Either can be *redirected* to come from or go to a file, which is how Seashell and Marmoset test our programs.

7.2 Output with `printf`

We have already used `printf` with the placeholders `%d`, `%c`, `%f`, `%zd` and `%p` (and `%s` for strings in Unit 08). Some details:

- The number of placeholders must match the number (and types) of extra arguments. A mismatch is undefined behaviour, but the compiler usually warns.
- `printf` produces output as a side effect. (It also returns the number of characters printed, which we rarely use.)
- Output is often *buffered*: it may not appear until a newline is printed or the program ends. If a program crashes, the last few lines of output can be lost, which can be misleading when debugging.

7.3 Debugging Output and Tracing

Tracing means printing intermediate values to see what a program is doing. To avoid having to delete tracing code before submitting, wrap it in a flag:

```
const bool DEBUG = true;   // set to false before submitting

int sum_first(int n) {
  int sum = 0;
```

```
  for (int i = 1; i <= n; ++i) {
    sum += i;
    if (DEBUG) {
      printf("TRACE: i = %d, sum = %d\n", i, sum);
    }
  }
  return sum;
}
```

A preprocessor version (`#ifdef DEBUG ... #endif`) removes the code completely at compile time. Printing traces to `stderr` with `fprintf(stderr, ...)` keeps them separate from the program's real output.

7.4 Input with `scanf`

`scanf` reads formatted input. It uses placeholders like `printf`, but each extra argument must be a *pointer* to the variable that will receive the value.

```
int n = 0;
int count = scanf("%d", &n);   // note the &
```

The return value of `scanf`

`scanf` returns the number of values successfully read and stored:

- `1` (for one placeholder): success;
- `0`: the input was not in the expected format (e.g. a letter when reading an int); the offending characters are left unread;
- `EOF` (a negative constant): the end of the input was reached before anything could be read.

Always check the return value. Never assume `scanf` succeeded.

End of file (EOF) is a special signal that there is no more input. When input comes from a file it happens at the end of the file; when typing in Seashell you can send it with **Ctrl+D**.

7.4.1 Whitespace

- `%d` (and `%f`, `%s`) skip leading whitespace (spaces, tabs, newlines) automatically.
- `%c` does *not*: it reads the very next character, which may be a space or newline. To skip whitespace before reading a character use `" %c"` (with a space).

7.4.2 Reading all integers until EOF

```
// read_sum() reads integers from input until EOF (or invalid input)
//   and produces their sum
// effects: reads input
```

```c
int read_sum(void) {
  int sum = 0;
  int n = 0;
  while (scanf("%d", &n) == 1) {
    sum += n;
  }
  return sum;
}
```

7.4.3 Reading commands

Many assignments read a sequence of commands. A typical structure:

```c
int main(void) {
  char cmd = 0;
  int arg = 0;
  while (scanf(" %c", &cmd) == 1) {
    if (cmd == 'a') {            // "a 5" adds 5
      if (scanf("%d", &arg) != 1) {
        printf("invalid input\n");
        break;
      }
      // ... add arg ...
    } else if (cmd == 'p') {     // "p" prints
      // ... print ...
    } else if (cmd == 'q') {     // "q" quits
      break;
    } else {
      printf("unknown command %c\n", cmd);
    }
  }
}
```

7.4.4 Reading characters with `getchar`

`getchar()` (in `<stdio.h>`) reads one character and returns it as an `int`, or `EOF`. The result must be stored in an `int`, not a `char`, so that `EOF` can be distinguished from every valid character.

```c
// count_lines() counts the lines of input
int count_lines(void) {
  int count = 0;
  int c = 0;
  while ((c = getchar()) != EOF) {
    if (c == '\n') ++count;
  }
  return count;
}
```

7.4.5 Invalid input

When `scanf("%d", &n)` returns 0, the bad characters are still waiting in the input, so calling it again in a loop would loop forever. Either stop, or skip the bad input (e.g. read characters with

`getchar` until a newline). Unless an assignment says otherwise, you may assume input is valid, but your loops should still terminate on EOF.

7.5 Testing

7.5.1 Why test?

Testing cannot prove a program correct, but good tests find most bugs. In CS 136, public tests on Marmoset only check basic functionality; private tests are much more thorough, so your own testing determines your mark.

7.5.2 Kinds of testing

- **Black-box testing:** tests are written from the specification (documentation) alone, without looking at the code. Good for checking the interface is correct.
- **White-box testing:** tests are written while looking at the code, to make sure every branch, loop and special case is exercised.
- **Assertion-based testing:** a test client calls functions and `assert`s the results (no I/O required). Good for modules.
- **I/O-driven testing:** the program reads input and produces output; tests compare the actual output with the expected output.
- **Regression testing:** re-running all old tests after every change to make sure nothing that used to work is now broken.

7.5.3 What to test

- **Boundary / edge cases:** 0, 1, negative numbers, `INT_MAX`, empty input, an empty collection, a collection with exactly one item, the first and last positions.
- **Each branch** of every conditional, and loops that run 0, 1 and many times.
- **Typical cases** (to check general correctness).
- **Large cases** (to check efficiency and stack depth).
- **Mutation:** check the state after each side effect, not only the returned value.

7.5.4 Testing in Seashell

For a program `prog.c` that reads input, create a pair of files in the `tests` folder:

- `name.in`: the input to feed to the program;
- `name.expect`: the exact expected output.

Running tests feeds each `.in` file to the program and reports whether the output matched the `.expect` file exactly (whitespace matters, including the final newline). This is the same mechanism Marmoset uses. Outside Seashell you would do the same with redirection:

```
./prog < test1.in > test1.out
diff test1.out test1.expect
```

7.5.5 Testing modules: a test client

```c
// test-stack.c: assertion-based tests for the stack module
#include <assert.h>
#include "stack.h"

int main(void) {
  struct stack s;
  stack_init(&s);
  assert(stack_is_empty(&s));       // empty case
  stack_push(&s, 1);
  assert(!stack_is_empty(&s));
  assert(stack_top(&s) == 1);       // one item
  stack_push(&s, 2);
  stack_push(&s, 3);
  assert(stack_pop(&s) == 3);       // LIFO order
  assert(stack_pop(&s) == 2);
  assert(stack_pop(&s) == 1);
  assert(stack_is_empty(&s));       // back to empty
}
```

An I/O-driven test harness is also common: `main` reads commands like `push 5` / `pop` / `top` and prints results, so many tests can be written as `.in`/`.expect` files without recompiling.

7.5.6 Debugging strategies

- Read the error message: assertion failures give the file and line.
- Make the failing test as small as possible.
- Add tracing output (with a `DEBUG` flag) around the suspected code.
- Trace the memory model by hand: draw the stack frames and pointers.
- Check the usual suspects: uninitialized variables, off-by-one loop bounds, `=` vs `==`, missing `&` in `scanf`, integer division.

7.6 Summary

- `printf` writes to stdout; use a `DEBUG` flag for tracing output.
- `scanf` needs pointers and returns the number of items read (or `EOF`); always check it. `" %c"` skips whitespace.
- Ctrl+D sends EOF in Seashell; `getchar` returns an `int` so `EOF` fits.
- Test black-box and white-box; cover edge cases; use `.in`/`.expect` files for I/O tests and `assert`-based test clients for modules; re-run all tests after changes.

Chapter 8

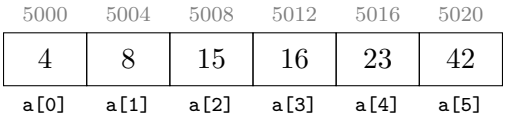
Arrays and Strings

Readings: CP:AMA 8, 12, 13.

8.1 Arrays

Definition 8.1. An *array* is a data structure containing a fixed number of elements, all of the same type, stored contiguously (one after another) in memory.

```
int a[6] = {4, 8, 15, 16, 23, 42};
```



Elements are accessed with square brackets: `a[0]` is the first element and `a[5]` the last. Array elements are mutable (unless the array is `const`): `a[0] = 7;`.

No bounds checking

C does **not** check array indices. `a[6]` or `a[-1]` silently accesses memory outside the array: undefined behaviour (corrupting other data, or a segmentation fault).

8.1.1 Initialization

Arrays can only be initialized with braces `{}`; they cannot be assigned as a whole after definition (`a = b;` is an error).

```
int b[5] = {1, 2};          // remaining elements are 0: {1, 2, 0, 0, 0}
int c[5] = {0};             // all zero
int d[] = {1, 2, 3};        // length 3 inferred
int e[5];                   // local: uninitialized (garbage)!
int f[5] = {[2] = 7};       // C99 designated: {0, 0, 7, 0, 0}
```

8.1.2 Length and size

The *length* of an array is its number of elements n; its *size* is $n \times$ `sizeof`(element) bytes. C does not store the length anywhere. Within the scope where an array is defined, the length can be computed with `sizeof(a) / sizeof(a[0])`, but this does *not* work once the array is passed to a function. So we keep track of the length ourselves, often with a constant:

```
const int LEN = 6;   // in C99 local arrays may use a variable length
int a[6] = {0};
```

8.2 Arrays and Pointers

An array is (almost) a pointer to its first element

The value of an array identifier `a` is the address of its first element, `&a[0]`. So `a` can be used wherever a pointer to the element type is expected, and `a[i]` is defined to mean `*(a + i)`.

8.2.1 Passing arrays to functions

When an array is passed to a function, only the address of the first element is copied (not the whole array), so the function must also be given the length. The function can mutate the caller's array; use `const` if it should not.

```
// sum_array(a, len) produces the sum of the elements of a
// requires: len >= 1
int sum_array(const int a[], int len) {    // same as const int *a
  int sum = 0;
  for (int i = 0; i < len; ++i) {
    sum += a[i];
  }
  return sum;
}

// array_negate(a, len) negates every element of a
// effects: modifies a
void array_negate(int a[], int len) {
  for (int i = 0; i < len; ++i) {
    a[i] = -a[i];
  }
}
```

8.3 Pointer Arithmetic

1. Adding an integer `i` to a pointer `p` gives the address of the element `i` positions later. In "normal" arithmetic the address is $\texttt{p} + \texttt{i} \times \texttt{sizeof(*p)}$.
2. Subtracting an integer (`p - i`) works in the same way.
3. Mutable pointers can be incremented and decremented: `++p` is `p = p + 1`.
4. You cannot add two pointers.

5. You can subtract a pointer `q` from `p` if they have the same type (and point into the same array). The result is the number of *elements* between them: $(\texttt{p} - \texttt{q})/\texttt{sizeof(*p)}$ in normal arithmetic. In other words, if `p == q + i` then `p - q == i`.

6. Pointers of the same type can be compared with `<`, `<=`, `==`, `!=`, `>=`, `>`.

So `a + 1` is `&a[1]`, and `*(a + 1)` is `a[1]`. Here is the same function written with pointer arithmetic:

```
int sum_array(const int *a, int len) {
  int sum = 0;
  for (const int *p = a; p < a + len; ++p) {
    sum += *p;
  }
  return sum;
}
```

Both styles are equivalent; array indexing is usually clearer.

8.4 Array "Map"

Applying a function to every element of an array is the array equivalent of Racket's `map`. With a function pointer:

```
// array_map(f, a, len) replaces each element a[i] with f(a[i])
// effects: modifies a
void array_map(int (*f)(int), int a[], int len) {
  for (int i = 0; i < len; ++i) {
    a[i] = f(a[i]);
  }
}

int a[4] = {1, 2, 3, 4};
array_map(my_sqr, a, 4);   // a is {1, 4, 9, 16}
```

Unlike Racket's `map`, this mutates the array in place instead of producing a new one.

8.5 Sorting

Sorting is a fundamental problem; here are three classic algorithms (efficiency in Unit 09). All use the `swap` function from Unit 06.

8.5.1 Selection sort

Select the smallest element and swap it into position 0, then the smallest of the rest into position 1, and so on.

```
void selection_sort(int a[], int len) {
  for (int i = 0; i < len - 1; ++i) {
    int pos = i;
    for (int j = i + 1; j < len; ++j) {
      if (a[j] < a[pos]) {
        pos = j;
```

```
      }
    }
    swap(&a[i], &a[pos]);
  }
}
// i:   loops from 0 ... len-2; the position to be filled next
// j:   loops from i+1 ... len-1, searching for the next smallest element
// pos: position of the smallest element seen so far
```

8.5.2 Insertion sort

Keep a sorted prefix `a[0..i-1]`; insert `a[i]` into it by swapping it backwards until it is in place (just like sorting a hand of cards).

```
void insertion_sort(int a[], int len) {
  for (int i = 1; i < len; ++i) {
    for (int j = i; j > 0 && a[j - 1] > a[j]; --j) {
      swap(&a[j], &a[j - 1]);
    }
  }
}
```

Insertion sort is very fast on arrays that are already (nearly) sorted.

8.5.3 Quicksort

Quicksort (C. A. R. Hoare) is a divide-and-conquer algorithm: choose a *pivot*, partition the array so that smaller elements come before it and larger ones after it, then recursively sort the two parts. In Racket we built new lists with `filter`; in C we partition in place.

```
// quick_sort_range(a, first, last) sorts a[first..last] (inclusive)
static void quick_sort_range(int a[], int first, int last) {
  if (last <= first) return;           // 0 or 1 elements
  int pivot = a[first];                // first element as pivot
  int pos = last;                      // where the next "big" element goes
  for (int i = last; i > first; --i) {
    if (a[i] > pivot) {
      swap(&a[pos], &a[i]);
      --pos;
    }
  }
  swap(&a[first], &a[pos]);            // put the pivot in its final place
  quick_sort_range(a, first, pos - 1);
  quick_sort_range(a, pos + 1, last);
}

void quick_sort(int a[], int len) {
  quick_sort_range(a, 0, len - 1);
}
```

The C library provides a generic `qsort` in `<stdlib.h>` (Unit 13).

8.6 Binary Search

Searching an unsorted array for a value requires checking every element (linear search). If the array is *sorted*, we can compare with the middle element and discard half of the remaining range each step.

```
// find_sorted(item, a, len) produces the index of item in a, or -1
// requires: a is sorted in increasing order
int find_sorted(int item, const int a[], int len) {
  int low = 0;
  int high = len - 1;
  while (low <= high) {
    int mid = low + (high - low) / 2;   // avoids overflow of low + high
    if (a[mid] == item) {
      return mid;
    } else if (a[mid] < item) {
      low = mid + 1;
    } else {
      high = mid - 1;
    }
  }
  return -1;
}
```

8.7 Multi-dimensional Arrays

```
int m[2][3] = {{1, 2, 3},
               {4, 5, 6}};
m[1][2] = 7;
```

C stores multi-dimensional arrays in *row-major order*: `m` is laid out as `1 2 3 4 5 6`, so `m[r][c]` is at offset `r * 3 + c`. When passing a 2D array to a function, every dimension except the first must be given (`int f(int m[][3], int rows)`). Often it is simpler to use a one-dimensional array and compute `a[r * cols + c]` yourself.

8.8 Fixed-Length Array Implementation of a Stack

```
#include <assert.h>
#include <stdbool.h>

#define MAXLEN 100

struct stack {
  int len;
  int maxlen;
  int data[MAXLEN];
};

// stack_init(s) initializes *s to an empty stack
void stack_init(struct stack *s) {
```

```
  assert(s);
  s->len = 0;
  s->maxlen = MAXLEN;
}

bool stack_is_empty(const struct stack *s) {
  assert(s);
  return s->len == 0;
}

// requires: stack is not empty
int stack_top(const struct stack *s) {
  assert(s);
  assert(s->len > 0);
  return s->data[s->len - 1];
}

// stack_pop(s) removes and produces the top element
// requires: stack is not empty
int stack_pop(struct stack *s) {
  assert(s);
  assert(s->len > 0);
  --s->len;
  return s->data[s->len];
}

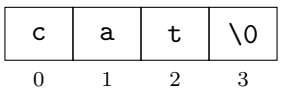
// requires: stack is not full
void stack_push(struct stack *s, int item) {
  assert(s);
  assert(s->len < s->maxlen);
  s->data[s->len] = item;
  ++s->len;
}
```

All operations are $\mathcal{O}(1)$, but the maximum size is fixed when the program is compiled. In Unit 10 we remove this limitation with dynamic memory.

8.9 Strings

C has no built-in string type. A *string* is an array of `char`s terminated by the *null character* `'\0'` (ASCII 0). The terminator is how functions know where the string ends, so the length does not need to be passed around.

```
char a[] = {'c', 'a', 't', '\0'};   // length 4 array, string length 3
char b[] = "cat";                   // the same, more conveniently
char c[10] = "cat";                 // remaining chars are '\0'
```



Null terminator

An array holding a string of length n needs at least $n + 1$ characters. Forgetting room for `'\0'`, or forgetting to add it, leads to functions reading past the end of the array.

8.9.1 String length

```
// my_strlen(s) produces the number of characters in s (excluding '\0')
int my_strlen(const char *s) {
  int len = 0;
  while (s[len] != '\0') {
    ++len;
  }
  return len;
}
```

8.9.2 String I/O

The placeholder for strings is `%s`. `printf("%s", s)` prints characters until the null terminator. `scanf("%s", s)` skips leading whitespace and then reads characters up to the next whitespace, adding a `'\0'`. Note that no `&` is needed: `s` is already a pointer.

Buffer overflow

`scanf("%s", name)` with `char name[10]` will happily write more than 10 characters if the user types a long word, overwriting whatever is next in memory. This is a *buffer overflow*, one of the most common security vulnerabilities. Imagine a bank program where an account balance is stored right after the name buffer: a long name changes the balance. Always give a maximum width: `scanf("%9s", name)` reads at most 9 characters (plus the `'\0'`).

8.9.3 `<string.h>` functions

- `strlen(s)`: the length of `s` (not counting `'\0'`).
- `strcmp(s1, s2)`: compares lexicographically (by ASCII values). Produces a *negative* number if `s1` comes before `s2`, zero if they are equal, and a *positive* number if `s1` comes after `s2`. (Not necessarily -1 and 1!)
- `strcpy(dest, src)`: copies `src` (including the terminator) into `dest`. `dest` must be large enough.
- `strcat(dest, src)`: appends `src` to the end of `dest`. `dest` must have room for both.

Comparing strings

`s1 == s2` compares the *addresses*, not the contents. Use `strcmp(s1, s2) == 0`.

A possible implementation of `strcmp`:

```c
int my_strcmp(const char *s1, const char *s2) {
  int i = 0;
  while (s1[i] == s2[i] && s1[i] != '\0') {
    ++i;
  }
  return s1[i] - s2[i];
}
```

8.9.4 String literals

For each string literal in the program (e.g. `"hello"` in `printf("hello")`), a null-terminated `const char` array is created in the read-only data section. For this reason the read-only section is also called the *literal pool*. Attempting to modify a string literal is undefined behaviour (often a segmentation fault).

8.10 Arrays vs. Pointers

```c
char a[] = "pointers are not arrays";
char *p  = "pointers are not arrays";
```

- `a` is an array: 24 bytes on the stack, initialized with a copy of the characters. Its contents can be changed (`a[0] = 'P';`), but `a` itself always refers to the same memory (`a = ...` is an error). `sizeof(a)` is 24.
- `p` is a pointer variable (8 bytes) pointing at a literal in the read-only section. `p` can be changed to point somewhere else, but `p[0] = 'P'` is undefined behaviour. `sizeof(p)` is 8. (It would be better declared `const char *p`.)

So arrays and pointers are closely related (an array "decays" to a pointer to its first element in most expressions) but they are not the same thing.

8.11 Summary

- Arrays are fixed-length, contiguous, unchecked; initialize with braces; pass with a length.
- `a[i]` means `*(a + i)`; pointer arithmetic counts in elements.
- Sorting: selection, insertion, quicksort; binary search on sorted arrays.
- 2D arrays are row-major.
- Strings are `'\0'`-terminated `char` arrays; use `<string.h>`; beware buffer overflow; compare with `strcmp`.
- String literals live in the read-only literal pool; arrays and pointers differ.

Chapter 9

Efficiency

Readings: none in CP:AMA (this material is not in the textbook).

9.1 Algorithms and Efficiency

Definition 9.1. An *algorithm* is a step-by-step description of how to solve a problem.

Different algorithms for the same problem can take very different amounts of resources.

- **Time efficiency:** how long it takes an algorithm to solve a problem (the most common measure).
- **Space efficiency:** how much memory it requires.

Unless stated otherwise, "efficiency" means time efficiency.

9.2 Running Time

Measuring with a stopwatch depends on the machine, the compiler and the input, so instead we count the number of *elementary operations* (arithmetic, comparisons, assignments, array indexing, function calls, ...), each of which takes constant time.

The running time depends on the size of the input, so it is a function $T(n)$, where n is the size of the input. What n means depends on the function: the length of an array, the number of nodes in a tree, the value of an integer argument, the length of a string. Other letters (such as m or k) are common when there are several inputs; always state what each variable means.

9.2.1 Worst case

The running time can depend on the *contents* of the input, not just its size (e.g. linear search finds the item immediately if it is first). We usually care about the **worst-case running time**: the maximum over all inputs of size n. It is a conservative guarantee: the function will never be slower than this. (Best case is rarely useful; average case is useful but harder to analyse.)

9.3 Big-O Notation

In practice we are interested in the *order* (growth rate) of the running time, not the exact constants. For example $17n + 1$ and $8n + 7$ are both "order n", written $\mathcal{O}(n)$.

Definition 9.2. $f(n) \in \mathcal{O}(g(n))$ if there exist constants $c > 0$ and $n_0 > 0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Example 9.3. $17n + 1 \in \mathcal{O}(n)$: take $c = 18$, $n_0 = 1$; then $17n + 1 \leq 18n$ for all $n \geq 1$. Also $17n + 1 \in \mathcal{O}(n^2)$, but that is not a useful (tight) bound. In CS 136 we always give the tightest simple bound.

9.3.1 Rules for simplifying

- Drop constant factors: $\mathcal{O}(5n) = \mathcal{O}(n)$.
- Keep only the dominant term: $\mathcal{O}(3n^2 + 100n + 7) = \mathcal{O}(n^2)$.
- Sum: $\mathcal{O}(f) + \mathcal{O}(g) = \mathcal{O}(\max(f, g))$ (sequential code).
- Product: $\mathcal{O}(f) \cdot \mathcal{O}(g) = \mathcal{O}(f \cdot g)$ (nested loops).
- Bases of logarithms do not matter: $\log_2 n$ and $\log_{10} n$ differ by a constant factor, so we just write $\mathcal{O}(\log n)$.

9.3.2 Common orders

Order	Name	Example
$\mathcal{O}(1)$	constant	array indexing, `first`
$\mathcal{O}(\log n)$	logarithmic	binary search
$\mathcal{O}(n)$	linear	linear search, `strlen`
$\mathcal{O}(n \log n)$	linearithmic	merge sort
$\mathcal{O}(n^2)$	quadratic	selection sort
$\mathcal{O}(n^3)$	cubic	naive matrix multiplication
$\mathcal{O}(2^n)$	exponential	naive Fibonacci

Each is strictly worse than the one above it for large n. For $n = 10^6$: $\log n \approx 20$, $n \log n \approx 2 \times 10^7$ (instant), $n^2 = 10^{12}$ (many minutes), and 2^n is hopeless.

> $\mathcal{O}$ is an *upper* bound. CS 240 introduces Ω (lower bound) and Θ (tight bound). When CS 136 says "the running time is $\mathcal{O}(n)$" it really means $\Theta(n)$.

9.4 Analysing Iterative Code

1. Each elementary statement is $\mathcal{O}(1)$.
2. A sequence of blocks takes the sum (i.e. the maximum) of their times.
3. A loop takes (number of iterations) $\times$ (time per iteration), or, if the time per iteration varies, the sum over all iterations.
4. A function call takes the running time of the function (not $\mathcal{O}(1)$!).

Example 9.4.

```
for (int i = 0; i < n; ++i) {          // n iterations
  for (int j = 0; j < n; ++j) {        //   n iterations each
    sum += i * j;                      //     O(1)
  }
}
```

$n \cdot n \cdot \mathcal{O}(1) = \mathcal{O}(n^2)$.

Example 9.5 (Selection sort)**.** The inner loop runs $n-1-i$ times for $i = 0, \ldots, n-2$, so the total is

$$\sum_{i=0}^{n-2}(n-1-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} \in \mathcal{O}(n^2).$$

Selection sort is $\mathcal{O}(n^2)$ in *every* case.

Example 9.6 (Halving loop)**.**

```
while (n > 1) {
  n /= 2;
  ++count;
}
```

n is halved each iteration, so there are about $\log_2 n$ iterations: $\mathcal{O}(\log n)$. This is why binary search is $\mathcal{O}(\log n)$.

Hidden costs

Watch out for function calls in loop conditions:

```
for (int i = 0; i < strlen(s); ++i) { ... }   // O(n^2)! strlen is O(n)
```

Compute `strlen(s)` once before the loop to make it $\mathcal{O}(n)$.

9.5 Analysing Recursive Code

For a recursive function, write a *recurrence relation* for $T(n)$: the time for the non-recursive work plus the time of the recursive calls. Then look it up in the table (you are expected to know these; deriving them is covered in later courses).

Recurrence	Solution
$T(n) = \mathcal{O}(1) + T(n-k)$	$\mathcal{O}(n)$
$T(n) = \mathcal{O}(n) + T(n-k)$	$\mathcal{O}(n^2)$
$T(n) = \mathcal{O}(n^2) + T(n-k)$	$\mathcal{O}(n^3)$
$T(n) = \mathcal{O}(1) + T(n/k)$	$\mathcal{O}(\log n)$
$T(n) = \mathcal{O}(1) + k \cdot T(n/k)$	$\mathcal{O}(n)$
$T(n) = \mathcal{O}(n) + k \cdot T(n/k)$	$\mathcal{O}(n \log n)$
$T(n) = \mathcal{O}(1) + T(n-k_1) + T(n-k_2)$	$\mathcal{O}(2^n)$

(for constants $k, k_1, k_2 \geq 1$; for the n/k rows, $k \geq 2$).

Example 9.7.

- `sum_first(n)` (Unit 02) does $\mathcal{O}(1)$ work and recurses on $n-1$: $\mathcal{O}(n)$.

- Recursive binary search: $T(n) = \mathcal{O}(1) + T(n/2)$: $\mathcal{O}(\log n)$.
- Naive `fib(n) = fib(n-1) + fib(n-2)`: $\mathcal{O}(2^n)$.
- A Racket function that calls `append` or `length` ($\mathcal{O}(n)$) on the rest of the list at each step: $T(n) = \mathcal{O}(n) + T(n-1) = \mathcal{O}(n^2)$.

9.6 Sorting Algorithms Revisited

9.6.1 Insertion sort

In the worst case (reverse-sorted input) inserting element i takes i swaps, so the total is $1 + 2 + \cdots + (n-1) \in \mathcal{O}(n^2)$. In the best case (already sorted) each insertion is $\mathcal{O}(1)$, giving $\mathcal{O}(n)$.

9.6.2 Quicksort

Partitioning is $\mathcal{O}(n)$. If the pivot always splits the array in half, $T(n) = \mathcal{O}(n) + 2T(n/2) = \mathcal{O}(n \log n)$. But in the worst case (e.g. an already sorted array with the first element as pivot) one side is empty every time: $T(n) = \mathcal{O}(n) + T(n-1) = \mathcal{O}(n^2)$. Quicksort's *average* case is $\mathcal{O}(n \log n)$, and it is very fast in practice, especially with a random pivot.

9.6.3 Merge sort

Merge sort guarantees $\mathcal{O}(n \log n)$: split the array into halves, sort each half recursively, and *merge* the two sorted halves in $\mathcal{O}(n)$ time.

```
// merge(dest, src1, len1, src2, len2) merges the sorted arrays src1 and
//   src2 into dest
// requires: dest has length len1 + len2 and does not overlap src1/src2
void merge(int dest[], const int src1[], int len1,
           const int src2[], int len2) {
  int i = 0, j = 0, k = 0;
  while (i < len1 && j < len2) {
    if (src1[i] <= src2[j]) {
      dest[k++] = src1[i++];
    } else {
      dest[k++] = src2[j++];
    }
  }
  while (i < len1) dest[k++] = src1[i++];
  while (j < len2) dest[k++] = src2[j++];
}

void merge_sort(int a[], int len) {
  if (len <= 1) return;
  int llen = len / 2;
  int rlen = len - llen;
  int *left = malloc(llen * sizeof(int));    // see Unit 10
  int *right = malloc(rlen * sizeof(int));
  for (int i = 0; i < llen; ++i) left[i] = a[i];
  for (int i = 0; i < rlen; ++i) right[i] = a[llen + i];
  merge_sort(left, llen);
  merge_sort(right, rlen);
```

```
  merge(a, left, llen, right, rlen);
  free(left);
  free(right);
}
```

$T(n) = \mathcal{O}(n) + 2T(n/2) = \mathcal{O}(n \log n)$ in every case, at the cost of $\mathcal{O}(n)$ extra space.

Algorithm	Best	Worst	Extra space
Selection sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Insertion sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Quicksort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$ to $\mathcal{O}(n)$ stack
Merge sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$

9.7 Running Times of Library Functions

9.7.1 Racket

- $\mathcal{O}(1)$: `cons`, `cons?`, `empty`, `empty?`, `first`, `rest`, `second`, arithmetic on small numbers, `equal?` on simple values.
- $\mathcal{O}(n)$: `length`, `last`, `reverse`, `append` (in the length of the first list), `list-ref` (in the index), `member`.
- Abstract list functions (`map`, `filter`, `foldr`) are $\mathcal{O}(n)$ when the function they apply is $\mathcal{O}(1)$.
- Racket's `sort` is $\mathcal{O}(n \log n)$.

9.7.2 C

- All the `<string.h>` functions are $\mathcal{O}(n)$, where n is the length of the string. For `strcmp`, n is the length of the *shorter* string; for `strcat`, `strlen(dest) + strlen(src)`.
- `printf` and `scanf` are $\mathcal{O}(1)$ for numbers and characters, and $\mathcal{O}(n)$ for strings (`%s`) of length n.
- Array indexing and pointer arithmetic are $\mathcal{O}(1)$.
- `malloc` and `free` are $\mathcal{O}(1)$ for our purposes; `realloc` is $\mathcal{O}(n)$ where n is the size of the block (Unit 10).

9.8 Documenting Efficiency

From now on, the documentation of every function includes its time efficiency, with the meaning of each variable:

```
// find_sorted(item, a, len) produces the index of item in a, or -1
// requires: a is sorted in increasing order
// time: O(log n), where n is len
```

9.9 Space Efficiency

Space efficiency is analysed the same way: how much additional memory does the function need as a function of n? Remember that recursion uses stack space: a non-tail recursive function with depth n uses $\mathcal{O}(n)$ space even if it allocates nothing, whereas the equivalent loop uses $\mathcal{O}(1)$.

9.10 Summary

- Measure efficiency by counting elementary operations as a function of input size n; focus on the worst case.
- Big-O ignores constant factors and lower-order terms; know the common orders and their ranking.
- Loops: iterations $\times$ cost; nested loops multiply; watch hidden $\mathcal{O}(n)$ calls.
- Recursion: write a recurrence and use the table.
- Selection sort $\mathcal{O}(n^2)$; insertion sort $\mathcal{O}(n^2)$ worst, $\mathcal{O}(n)$ best; quicksort $\mathcal{O}(n \log n)$ average, $\mathcal{O}(n^2)$ worst; merge sort $\mathcal{O}(n \log n)$ always.
- Document *time:* in every contract.

Chapter 10

Dynamic Memory and ADTs in C

Readings: CP:AMA 17.1–17.4, 19.

10.1 Motivation

So far, the size of every piece of data had to be known when the program was written: arrays have a fixed length, and local variables disappear when their function returns. That causes problems:

- We cannot read an arbitrary number of values (how big should the array be?).
- A function cannot create data that outlives its own stack frame (returning a pointer to a local is a dangling pointer).
- An opaque structure cannot be created by a client, since the client does not know its size.

The solution is *dynamic memory*: memory requested while the program is running, from the **heap** section.

10.2 Duration (Lifetime)

Kind	Where	Lifetime
global/static	global or read-only data	the entire program
local (automatic)	stack	until its block/function ends
dynamic	heap	from `malloc` until `free`

10.3 `malloc`

`malloc` ("memory allocate", in `<stdlib.h>`) requests a block of memory from the heap:

```
void *malloc(size_t s);
```

It returns a pointer to the start of a block of (at least) `s` bytes, or `NULL` if there is not enough memory. The return type `void *` is a *generic pointer*: it can be assigned to any pointer type without a cast.

```c
#include <stdlib.h>

int *p = malloc(sizeof(int));          // one int
*p = 42;

int *arr = malloc(100 * sizeof(int));  // an array of 100 ints
arr[99] = 7;

struct posn *pp = malloc(sizeof(struct posn));
pp->x = 1;
```

Using `malloc` correctly

- Always use `sizeof` — never hard-code sizes like 4.
- The memory is **uninitialized** (garbage). Initialize it yourself, or use `calloc(n, size)`, which zero-fills.
- `malloc` can return `NULL`. In CS 136 we may assume it succeeds, but robust code checks.
- Every block that is `malloc`ed must eventually be `free`d exactly once.



10.4 `free`

```c
void free(void *p);
```

`free(p)` returns the block that `p` points at to the heap so it can be reused. `p` must be a pointer that was returned by `malloc` (or `calloc`/`realloc`) and not yet freed. `free(NULL)` does nothing.

10.4.1 Memory errors

- **Memory leak:** losing the last pointer to a block without freeing it. The memory cannot be used again until the program ends. Long-running programs that leak eventually run out of memory. Seashell reports leaks when the program exits; leaks lose marks.

```c
int *p = malloc(sizeof(int));
p = malloc(sizeof(int));     // leak: the first block is unreachable
```

- **Dangling pointer (use after free):** using a pointer after its block has been freed. The memory may have been reused for something else.

```c
free(p);
*p = 5;          // undefined behaviour
p = NULL;        // good habit: then *p crashes immediately instead
```

- **Double free:** freeing the same block twice (often through two aliases): undefined behaviour, typically corrupting the heap.
- Freeing memory that was not `malloc`ed (e.g. `free(&x)` on a local) is also undefined behaviour.

> The heap does not have to be used in order: blocks can be freed in any order, which leaves "holes" (*fragmentation*). The heap manager keeps track of free blocks. On Linux, `valgrind` is a common tool to detect leaks and invalid accesses.

10.5 Documenting Dynamic Memory

If a function returns memory that the caller must free, that is part of the interface and must be documented:

```c
// make_range(n) produces a new array {0, 1, ..., n-1}
// requires: n > 0
// effects: allocates memory (caller must free)
// time: O(n)
int *make_range(int n) {
  assert(n > 0);
  int *a = malloc(n * sizeof(int));
  for (int i = 0; i < n; ++i) {
    a[i] = i;
  }
  return a;
}

int main(void) {
  int *r = make_range(10);
  // ... use r ...
  free(r);
}
```

Unlike returning the address of a local, returning a heap pointer is fine: the block survives until it is freed.

10.6 `realloc`

```c
void *realloc(void *p, size_t newsize);
```

`realloc` resizes a previously allocated block to `newsize` bytes and returns a pointer to the (possibly moved) block. The old contents are preserved (up to the smaller of the two sizes). If it moves the block, it copies the data and frees the old block, so the old pointer must no longer be used.

- `realloc(NULL, s)` behaves like `malloc(s)`.
- Always use the returned pointer: `a = realloc(a, new_len * sizeof(int));`
- In the worst case realloc must copy the whole block, so it is $\mathcal{O}(n)$ where n is the size of the block.

10.6.1 Reading a string of unknown length

```c
// read_line() reads a line of input (without the '\n') into a new string
//   or produces NULL at EOF
// effects: reads input, allocates memory (caller must free)
// time: O(n), where n is the length of the line
char *read_line(void) {
  int maxlen = 1;
  int len = 0;
  char *str = malloc(maxlen * sizeof(char));
  int c = getchar();
  if (c == EOF) {
    free(str);
    return NULL;
  }
  while (c != EOF && c != '\n') {
    if (len + 1 == maxlen) {        // need room for c and '\0'
      maxlen *= 2;
      str = realloc(str, maxlen * sizeof(char));
    }
    str[len] = c;
    ++len;
    c = getchar();
  }
  str[len] = '\0';
  return str;
}
```

10.7 Dynamic Arrays and Amortized Analysis

A *dynamic array* (resizable array) keeps a pointer to a heap array, together with the current length and the allocated capacity:

```c
struct dynarray {
  int *data;
  int len;       // number of elements in use
  int maxlen;    // allocated capacity
};
```

When it is full and we want to append, we must grow it. How much should it grow?

Grow by one. Each append calls `realloc`, which may copy all k existing elements. Appending n elements costs $1 + 2 + \cdots + n \in \mathcal{O}(n^2)$ — $\mathcal{O}(n)$ per append.

Double the capacity. When full, double `maxlen`. Most appends are $\mathcal{O}(1)$; an expensive copy happens only when the length reaches $1, 2, 4, 8, \ldots$. Appending n elements copies at most

$$1 + 2 + 4 + \cdots + 2^{\lfloor \log_2 n \rfloor} < 2n$$

elements in total, plus $\mathcal{O}(1)$ work per append, so the total is $\mathcal{O}(n)$.

Definition 10.1. The *amortized* running time of an operation is the total time of a sequence of n operations divided by n. With doubling, append is $\mathcal{O}(1)$ *amortized*, even though a single append can take $\mathcal{O}(n)$ time.

Doubling strategy

Grow a dynamic array by a constant *factor* (doubling), not by a constant *amount*. Doubling makes n appends $\mathcal{O}(n)$ in total ($\mathcal{O}(1)$ amortized each), at the cost of up to half the allocated memory being unused.

Shrinking works similarly: halve the capacity when the array becomes only a quarter full (not half — otherwise alternating append/remove at the boundary would resize every time).

10.8 ADTs in C

Now we can build proper ADTs in C, combining:

- an **opaque structure** declared in the header (clients can only hold pointers);
- **create** and **destroy** functions that allocate and free the structure;
- interface functions that take a pointer to the structure.

10.8.1 Interface: `stack.h`

```c
// stack.h: a stack ADT of integers
#include <stdbool.h>

struct stack;   // opaque

// stack_create() creates a new empty stack
// effects: allocates memory (client must call stack_destroy)
// time: O(1)
struct stack *stack_create(void);

// stack_destroy(s) frees all memory used by s
// effects: s is no longer valid
// time: O(1)
void stack_destroy(struct stack *s);

// time: O(1)
bool stack_is_empty(const struct stack *s);

// requires: s is not empty
// time: O(1)
int stack_top(const struct stack *s);

// stack_pop(s) removes and produces the top item
// requires: s is not empty
// effects: modifies s
// time: O(1)
```

```c
int stack_pop(struct stack *s);

// effects: modifies s
// time: O(1) amortized
void stack_push(int item, struct stack *s);
```

10.8.2 Implementation: `stack.c`

```c
#include <assert.h>
#include <stdlib.h>
#include "stack.h"

struct stack {
  int len;
  int maxlen;
  int *data;
};

struct stack *stack_create(void) {
  struct stack *s = malloc(sizeof(struct stack));
  s->len = 0;
  s->maxlen = 1;
  s->data = malloc(s->maxlen * sizeof(int));
  return s;
}

void stack_destroy(struct stack *s) {
  assert(s);
  free(s->data);     // free the inner array first...
  free(s);           // ...then the structure itself
}

bool stack_is_empty(const struct stack *s) {
  assert(s);
  return s->len == 0;
}

int stack_top(const struct stack *s) {
  assert(s);
  assert(s->len > 0);
  return s->data[s->len - 1];
}

int stack_pop(struct stack *s) {
  assert(s);
  assert(s->len > 0);
  --s->len;
  return s->data[s->len];
}

void stack_push(int item, struct stack *s) {
  assert(s);
  if (s->len == s->maxlen) {
```

```
    s->maxlen *= 2;
    s->data = realloc(s->data, s->maxlen * sizeof(int));
  }
  s->data[s->len] = item;
  ++s->len;
}
```

10.8.3 Client

```
#include <stdio.h>
#include "stack.h"

int main(void) {
  struct stack *s = stack_create();
  for (int i = 0; i < 1000; ++i) {
    stack_push(i, s);              // no fixed maximum any more
  }
  while (!stack_is_empty(s)) {
    printf("%d\n", stack_pop(s));
  }
  stack_destroy(s);
}
```

The client cannot write `s->len` or `struct stack x;` because the structure is not defined in `stack.h`; the compiler enforces the abstraction. We can later replace the dynamic array by a linked list (Unit 11) without changing any client.

Freeing nested structures

Free memory "inside out": free what a structure points to before freeing the structure itself. Freeing `s` first would make `s->data` a dangling access and leak the array.

10.9 Summary

- The heap holds dynamic memory whose lifetime is controlled by the programmer.
- `malloc(n * sizeof(T))` allocates (uninitialized); `free` releases exactly once; `realloc` resizes (and may move, $\mathcal{O}(n)$).
- Errors: leaks, dangling pointers, double frees.
- Document "allocates memory (caller must free)" in the interface.
- Dynamic arrays with doubling give $\mathcal{O}(1)$ amortized append.
- C ADTs: opaque struct + create/destroy + interface functions taking pointers.

Chapter 11

Linked Data Structures

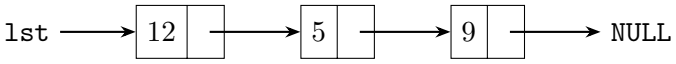
Readings: CP:AMA 17.5–17.6.

11.1 Linked Lists

Racket lists are *linked*: each `cons` holds an item and a reference to the rest of the list. In C we build them ourselves with a self-referential structure:

```
struct llnode {
  int item;
  struct llnode *next;    // NULL marks the end of the list
};
```

A structure cannot contain itself, but it can contain a *pointer* to its own type.



11.1.1 Building lists

```
// cons(first, rest) produces a new node with item first, followed by rest
// effects: allocates memory (caller must free, e.g. with list_destroy)
// time: O(1)
struct llnode *cons(int first, struct llnode *rest) {
  struct llnode *new = malloc(sizeof(struct llnode));
  new->item = first;
  new->next = rest;
  return new;
}

struct llnode *lst = cons(12, cons(5, cons(9, NULL)));
```

11.1.2 Traversal

Linked lists can be traversed recursively (like Racket) or iteratively:

```
// length(lst) produces the number of nodes in lst
// time: O(n)
int length(const struct llnode *lst) {
```

```
  int len = 0;
  while (lst) {           // same as lst != NULL
    ++len;
    lst = lst->next;      // lst is a local copy; the list is unchanged
  }
  return len;
}

int length_rec(const struct llnode *lst) {
  if (lst == NULL) return 0;
  return 1 + length_rec(lst->next);
}

// print_list(lst) prints the items of lst
void print_list(const struct llnode *lst) {
  for (const struct llnode *node = lst; node; node = node->next) {
    printf("%d ", node->item);
  }
  printf("\n");
}
```

The iterative version uses $\mathcal{O}(1)$ stack space, whereas the recursive one uses $\mathcal{O}(n)$ and can overflow on long lists.

11.1.3 Destroying a list

```
// list_destroy(lst) frees every node of lst
// time: O(n)
void list_destroy(struct llnode *lst) {
  while (lst) {
    struct llnode *next = lst->next;   // save before freeing!
    free(lst);
    lst = next;
  }
}
```

Writing `free(lst); lst = lst->next;` would use freed memory.

11.1.4 Mutating lists

Unlike Racket, we can change a list in place. Mapping in place:

```
void list_sqr(struct llnode *lst) {
  for (struct llnode *node = lst; node; node = node->next) {
    node->item *= node->item;
  }
}
```

Copying (like Racket's functional style) must allocate new nodes:

```
struct llnode *list_copy(const struct llnode *lst) {
  if (lst == NULL) return NULL;
  return cons(lst->item, list_copy(lst->next));
}
```

11.2 A List Wrapper Structure

Functions that add or remove the *first* node need to change the caller's pointer `lst`, which would require a `struct llnode **`. A cleaner approach is a wrapper structure that represents the whole list:

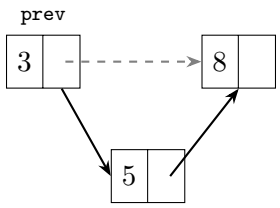
```
struct llist {
  struct llnode *front;
};

struct llist *list_create(void) {
  struct llist *lst = malloc(sizeof(struct llist));
  lst->front = NULL;
  return lst;
}

// add_front(i, lst) adds i to the front of lst
// time: O(1)
void add_front(int i, struct llist *lst) {
  lst->front = cons(i, lst->front);
}
```

11.2.1 Inserting into a sorted list

```
// insert(i, slst) inserts i into the sorted list slst
// requires: slst is sorted in increasing order
// time: O(n)
void insert(int i, struct llist *slst) {
  if (slst->front == NULL || i < slst->front->item) {
    add_front(i, slst);                 // special case: new first node
  } else {
    struct llnode *prev = slst->front;
    while (prev->next && i > prev->next->item) {
      prev = prev->next;
    }
    prev->next = cons(i, prev->next);   // splice in after prev
  }
}
```



11.2.2 Removing a node

```
// remove_item(i, lst) removes the first occurrence of i from lst
//   and produces true, or produces false if i is not in lst
// time: O(n)
```

```
bool remove_item(int i, struct llist *lst) {
  if (lst->front == NULL) return false;
  if (lst->front->item == i) {
    struct llnode *old = lst->front;
    lst->front = old->next;
    free(old);
    return true;
  }
  struct llnode *prev = lst->front;
  while (prev->next && prev->next->item != i) {
    prev = prev->next;
  }
  if (prev->next == NULL) return false;
  struct llnode *old = prev->next;
  prev->next = old->next;
  free(old);
  return true;
}
```

The pattern for both: handle the empty list and the front node as special cases, then walk a `prev` pointer to the node *before* the position of interest.

11.3 Augmenting the Wrapper

The wrapper can store extra information to make operations faster, as long as every function keeps it up to date:

- a `length` field makes `list_length` $\mathcal{O}(1)$ instead of $\mathcal{O}(n)$;
- a `back` pointer to the last node makes adding to the back $\mathcal{O}(1)$.

11.3.1 A queue with a linked list

```
struct queue {
  struct llnode *front;
  struct llnode *back;
};

// time: O(1)
void queue_add_back(int i, struct queue *q) {
  struct llnode *node = cons(i, NULL);
  if (q->front == NULL) {
    q->front = node;
  } else {
    q->back->next = node;
  }
  q->back = node;
}

// requires: q is not empty
// time: O(1)
int queue_remove_front(struct queue *q) {
  assert(q->front);
```

```
  struct llnode *old = q->front;
  int item = old->item;
  q->front = old->next;
  if (q->front == NULL) {
    q->back = NULL;            // the queue is now empty
  }
  free(old);
  return item;
}
```

11.4 Arrays vs. Linked Lists

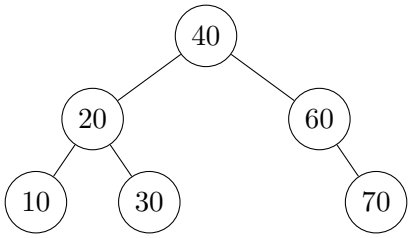
Operation	Dynamic array	Linked list
access item i	$\mathcal{O}(1)$	$\mathcal{O}(i)$
add/remove at front	$\mathcal{O}(n)$	$\mathcal{O}(1)$
add at back	$\mathcal{O}(1)$ amortized	$\mathcal{O}(1)$ with back pointer
insert/remove in middle (given position)	$\mathcal{O}(n)$	$\mathcal{O}(1)$ after reaching it
binary search (sorted)	$\mathcal{O}(\log n)$	not possible
extra memory	unused capacity	one pointer per item

11.5 Trees

A node in a binary tree has two child pointers:

```
struct bstnode {
  int item;
  struct bstnode *left;
  struct bstnode *right;
};
```

In a *binary search tree* (BST), every item in the left subtree is less than the node's item, and every item in the right subtree is greater.



11.5.1 BST operations

```
struct bst {
  struct bstnode *root;
};

// bst_find(i, t) determines if i is in t
// time: O(h), where h is the height of t
```

```
bool bst_find(int i, const struct bst *t) {
  const struct bstnode *node = t->root;
  while (node) {
    if (i == node->item) return true;
    node = (i < node->item) ? node->left : node->right;
  }
  return false;
}

static struct bstnode *new_leaf(int i) {
  struct bstnode *leaf = malloc(sizeof(struct bstnode));
  leaf->item = i;
  leaf->left = NULL;
  leaf->right = NULL;
  return leaf;
}

// bst_insert(i, t) inserts i into t (no effect if already present)
// time: O(h)
void bst_insert(int i, struct bst *t) {
  if (t->root == NULL) {
    t->root = new_leaf(i);
    return;
  }
  struct bstnode *node = t->root;
  while (true) {
    if (i == node->item) return;
    if (i < node->item) {
      if (node->left == NULL) { node->left = new_leaf(i); return; }
      node = node->left;
    } else {
      if (node->right == NULL) { node->right = new_leaf(i); return; }
      node = node->right;
    }
  }
}

// free_bstnode(node) frees every node in the subtree rooted at node
// time: O(n)
static void free_bstnode(struct bstnode *node) {
  if (node) {
    free_bstnode(node->left);
    free_bstnode(node->right);
    free(node);                    // post-order: children first
  }
}

void bst_destroy(struct bst *t) {
  free_bstnode(t->root);
  free(t);
}
```

A recursive insert is also natural; it returns the (possibly new) root of the subtree:

```
static struct bstnode *insert_node(int i, struct bstnode *node) {
```

```
  if (node == NULL) return new_leaf(i);
  if (i < node->item) {
    node->left = insert_node(i, node->left);
  } else if (i > node->item) {
    node->right = insert_node(i, node->right);
  }
  return node;
}
```

11.5.2 Height and balance

BST operations are $\mathcal{O}(h)$ where h is the height. A *balanced* tree (e.g. AVL trees, CS 240) has $h \in \mathcal{O}(\log n)$, giving $\mathcal{O}(\log n)$ operations. But inserting items in sorted order produces a "stick" with $h = n$, so in the worst case BST operations are $\mathcal{O}(n)$.

> Augmenting each node with the size of its subtree allows selecting the k-th smallest item in $\mathcal{O}(h)$ time — the same idea as augmenting the list wrapper with a length.

11.6 Graphs (briefly)

Linked structures can also represent general graphs, where nodes may have any number of neighbours and cycles are possible (e.g. an adjacency list: each node has a linked list of its neighbours). Cycles mean traversals must remember visited nodes, and freeing must be done carefully. Graph algorithms are covered in CS 341.

11.7 Summary

- A linked list node holds an item and a `next` pointer; `NULL` ends the list.
- Traverse with a moving pointer; save `next` before `free`.
- Use a wrapper structure to handle changes to the front; augment it with length/back for $\mathcal{O}(1)$ operations.
- Arrays give fast indexing; linked lists give fast insertion/removal at known positions.
- BST find/insert are $\mathcal{O}(h)$: $\mathcal{O}(\log n)$ if balanced, $\mathcal{O}(n)$ worst case. Free trees in post-order.

Chapter 12

Abstract Data Types in C

Readings: CP:AMA 19.3–19.5.

In Unit 03 we introduced ADTs as collections described only by their operations. Now that we have opaque structures, dynamic memory and linked data, we can implement them properly in C and compare implementations by efficiency.

12.1 Designing an ADT

Designing an ADT means deciding:

1. **The operations** and their exact behaviour (documented in the `.h` file).
2. **Error handling:** what happens on invalid use? Either make it a precondition ("requires: stack is not empty", checked with `assert`) or report it (return a `bool`, return `NULL`, a special value).
3. **Memory ownership:** who allocates and frees what? Does the ADT store a *copy* of the client's data (e.g. duplicate a string) or just the pointer it was given? Who frees items when the ADT is destroyed?
4. **The implementation**: the data structure and the *invariants* it maintains (e.g. "the array is sorted", "`len <= maxlen`", "BST ordering holds"), chosen for the efficiency the clients need.

Invariants

A *representation invariant* is a property of the internal data that every interface function may assume on entry and must restore before returning. Because the structure is opaque, only the module's own functions can break an invariant — this is exactly why information hiding gives *security*.

12.2 The Dictionary ADT

12.2.1 Interface

We implement a dictionary from `int` keys to string values. The dictionary stores its own copies of the strings.

```c
// dictionary.h: a dictionary ADT (int keys -> string values)
#include <stdbool.h>

struct dictionary;

// dict_create() creates a new, empty dictionary
// effects: allocates memory (client must call dict_destroy)
struct dictionary *dict_create(void);

// dict_destroy(d) frees all memory used by d (including stored strings)
void dict_destroy(struct dictionary *d);

// dict_insert(key, val, d) adds key -> val to d; if key is already
//   present, its value is replaced. d stores a copy of val.
// effects: modifies d
// time: O(h + m), h is the height of the tree, m is the length of val
void dict_insert(int key, const char *val, struct dictionary *d);

// dict_lookup(key, d) produces the value for key, or NULL if not present
//   the produced string must not be modified or freed by the client
// time: O(h)
const char *dict_lookup(int key, const struct dictionary *d);

// dict_remove(key, d) removes key from d (no effect if not present)
// effects: modifies d
// time: O(h)
void dict_remove(int key, struct dictionary *d);
```

12.2.2 Implementation with a BST

```c
// dictionary.c
#include <assert.h>
#include <stdlib.h>
#include <string.h>
#include "dictionary.h"

struct bstnode {
  int key;
  char *val;                    // owned by the dictionary
  struct bstnode *left;
  struct bstnode *right;
};

struct dictionary {
  struct bstnode *root;
};

// my_strdup(s) produces a new heap copy of s
static char *my_strdup(const char *s) {
  char *copy = malloc((strlen(s) + 1) * sizeof(char));   // +1 for '\0'
  strcpy(copy, s);
  return copy;
```

```
}

struct dictionary *dict_create(void) {
  struct dictionary *d = malloc(sizeof(struct dictionary));
  d->root = NULL;
  return d;
}

static void free_node(struct bstnode *node) {
  if (node) {
    free_node(node->left);
    free_node(node->right);
    free(node->val);
    free(node);
  }
}

void dict_destroy(struct dictionary *d) {
  assert(d);
  free_node(d->root);
  free(d);
}

static struct bstnode *insert_node(int key, const char *val,
                                   struct bstnode *node) {
  if (node == NULL) {
    struct bstnode *leaf = malloc(sizeof(struct bstnode));
    leaf->key = key;
    leaf->val = my_strdup(val);
    leaf->left = NULL;
    leaf->right = NULL;
    return leaf;
  }
  if (key < node->key) {
    node->left = insert_node(key, val, node->left);
  } else if (key > node->key) {
    node->right = insert_node(key, val, node->right);
  } else {                                    // replace existing value
    free(node->val);
    node->val = my_strdup(val);
  }
  return node;
}

void dict_insert(int key, const char *val, struct dictionary *d) {
  assert(d && val);
  d->root = insert_node(key, val, d->root);
}

const char *dict_lookup(int key, const struct dictionary *d) {
  assert(d);
  const struct bstnode *node = d->root;
  while (node) {
    if (key == node->key) return node->val;
```

```
    node = (key < node->key) ? node->left : node->right;
  }
  return NULL;
}
```

12.2.3 Removing from a BST

Removing a node has three cases:

1. **No children:** just free it (the parent's pointer becomes `NULL`).
2. **One child:** replace the node with its child.
3. **Two children:** find the *successor* (the smallest key in the right subtree, i.e. the leftmost node there), copy its key and value into the node, and then remove the successor from the right subtree (it has no left child, so that is case 1 or 2).

```
// remove_node(key, node) removes key from the subtree rooted at node
//   and produces the new root of that subtree
static struct bstnode *remove_node(int key, struct bstnode *node) {
  if (node == NULL) return NULL;            // key not found
  if (key < node->key) {
    node->left = remove_node(key, node->left);
  } else if (key > node->key) {
    node->right = remove_node(key, node->right);
  } else if (node->left == NULL || node->right == NULL) {
    struct bstnode *child = node->left ? node->left : node->right;
    free(node->val);
    free(node);
    return child;                           // cases 1 and 2
  } else {
    struct bstnode *succ = node->right;     // case 3
    while (succ->left) {
      succ = succ->left;
    }
    free(node->val);
    node->key = succ->key;
    node->val = my_strdup(succ->val);
    node->right = remove_node(succ->key, node->right);
  }
  return node;
}

void dict_remove(int key, struct dictionary *d) {
  assert(d);
  d->root = remove_node(key, d->root);
}
```

Instead of copying the successor's string and then freeing the original, we could move the pointer over and arrange for the recursive call not to free it. The version above is simpler to reason about.

12.2.4 Comparing dictionary implementations

Implementation	lookup	insert	remove
unsorted array / linked list	$\mathcal{O}(n)$	$\mathcal{O}(n)^*$	$\mathcal{O}(n)$
sorted array	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
BST (worst case)	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
balanced BST (AVL, CS 240)	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
hash table (CS 240)	$\mathcal{O}(1)$ average	$\mathcal{O}(1)$ average	$\mathcal{O}(1)$ average

*$\mathcal{O}(1)$ if duplicates are not checked (but then insert must search to replace a value).

The point of the ADT is that a client of `dictionary.h` does not change at all if we swap one implementation for another.

12.3 Stack, Queue and Sequence Implementations

12.3.1 Stack

Both a dynamic array (Unit 10) and a linked list (push/pop at the front) give $\mathcal{O}(1)$ operations (amortized for the array). The array version has better memory locality; the list version never needs to copy.

12.3.2 Queue

A linked list with front and back pointers (Unit 11) gives $\mathcal{O}(1)$ add-back and remove-front. An array-based queue would need $\mathcal{O}(n)$ to remove from the front by shifting, unless we use a *circular buffer*:

```
struct queue {
  int *data;
  int maxlen;
  int front;     // index of the front item
  int len;       // number of items
};

// time: O(1) amortized
void queue_add_back(int item, struct queue *q) {
  if (q->len == q->maxlen) {
    int *new_data = malloc(2 * q->maxlen * sizeof(int));
    for (int i = 0; i < q->len; ++i) {        // unwrap into the new array
      new_data[i] = q->data[(q->front + i) % q->maxlen];
    }
    free(q->data);
    q->data = new_data;
    q->front = 0;
    q->maxlen *= 2;
  }
  q->data[(q->front + q->len) % q->maxlen] = item;
  ++q->len;
}

// requires: q is not empty
```

```c
// time: O(1)
int queue_remove_front(struct queue *q) {
  assert(q->len > 0);
  int item = q->data[q->front];
  q->front = (q->front + 1) % q->maxlen;
  --q->len;
  return item;
}
```

The indices "wrap around" using `%`, so the items occupy a contiguous range modulo the capacity.

12.3.3 Sequence

Implementation	item_at	insert_at / remove_at	length
dynamic array	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
linked list (with length)	$\mathcal{O}(i)$	$\mathcal{O}(i)$	$\mathcal{O}(1)$
balanced tree with subtree sizes	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$

No implementation is best for every operation; the right choice depends on which operations the client uses most.

12.4 Set ADT

A *set* stores unique items with operations `member`, `add`, `remove`, and often `union`, `intersection`, `size`. It is essentially a dictionary with keys but no values, so the same implementations apply. With sorted arrays or sorted lists, union and intersection can be done in $\mathcal{O}(n+m)$ by a merge-like simultaneous walk.

12.5 Iterating Over an ADT

Clients often need to visit every item, but they cannot access the internal structure. Common solutions:

- an interface function that produces a new array of all items (in order), which the client must free;
- a "map"-style function that applies a client's function pointer to every item;
- an iterator interface (`first`, `next`, `done`), as in C++ (CS 246).

```c
// dict_keys(d, len) produces a new array of the keys of d in increasing
//   order and stores the number of keys in *len
// effects: allocates memory (caller must free), modifies *len
// time: O(n)
int *dict_keys(const struct dictionary *d, int *len);
```

12.6 Summary

- ADT design: operations, error handling (requires vs. return codes), memory ownership, and implementation invariants.
- Dictionary via BST: lookup/insert/remove are $\mathcal{O}(h)$; removal with two children uses the successor.
- Compare implementations in a table; choose based on which operations matter.
- Queues can use a linked list or a circular array; sequences trade indexing against insertion.
- Clients can iterate through an exported array, a map with function pointers, or an iterator.

Chapter 13

Beyond CS 136

Readings: CP:AMA 17.7 (function pointers), 2.1 and 15.4 (compiling and linking), 26.2 (`qsort`).

> Much of this unit is a preview of later courses (CS 246, CS 241, CS 240) rather than core exam material; check with your instructor which parts are tested. Generic `void *` ADTs and function pointers were the parts we actually used.

13.1 Generic Data with `void *`

All our ADTs so far store `int`s. To store strings we would copy the whole module and change the types. Racket does not have this problem because it is dynamically typed. In C, the *generic pointer* `void *` can point to data of any type:

- any pointer can be assigned to a `void *` and back without a cast;
- a `void *` cannot be dereferenced (the type, and hence the size, is unknown) and pointer arithmetic on it is not allowed;
- the compiler can no longer check types, so the client must be careful.

13.1.1 A generic stack

```
// gstack.h: a generic stack; it stores pointers provided by the client.
//   The client owns the items: the stack never frees them.
struct stack;
struct stack *stack_create(void);
void stack_destroy(struct stack *s);   // requires: s is empty
bool stack_is_empty(const struct stack *s);
void stack_push(void *item, struct stack *s);
void *stack_top(const struct stack *s);
void *stack_pop(struct stack *s);
```

```
// gstack.c (dynamic array of void *)
struct stack {
  int len;
  int maxlen;
  void **data;              // array of generic pointers
```

```
};

void stack_push(void *item, struct stack *s) {
  assert(s);
  if (s->len == s->maxlen) {
    s->maxlen *= 2;
    s->data = realloc(s->data, s->maxlen * sizeof(void *));
  }
  s->data[s->len] = item;
  ++s->len;
}
```

```
// client: a stack of heap-allocated ints
struct stack *s = stack_create();
int *p = malloc(sizeof(int));
*p = 42;
stack_push(p, s);
int *q = stack_pop(s);        // void * converted back to int *
printf("%d\n", *q);           // 42
free(q);                      // the client frees its own items
stack_destroy(s);
```

Ownership of generic items

With `void *` ADTs the interface must say who frees the items. Common choices: the client frees them (as above), or the client passes a "destroy" function pointer to `stack_destroy` so the ADT can free any remaining items.

13.2 Function Pointers for Generic Algorithms

A generic data structure that needs to *compare* items (a sorted list, a BST dictionary) cannot use `<` on `void *`. The client provides a comparison function instead, using the same convention as `strcmp`: negative, zero or positive.

13.2.1 `qsort`

The C library's generic sort (in `<stdlib.h>`):

```
void qsort(void *base, size_t len, size_t size,
           int (*compare)(const void *, const void *));
```

```
// compare_ints(a, b) compares the ints a and b point at
int compare_ints(const void *a, const void *b) {
  const int *ia = a;
  const int *ib = b;
  if (*ia < *ib) return -1;
  if (*ia > *ib) return 1;
  return 0;
  // (return *ia - *ib; is shorter but can overflow)
}
```

```
int a[5] = {5, 3, 9, 1, 7};
qsort(a, 5, sizeof(int), compare_ints);   // a is {1, 3, 5, 7, 9}
```

Sorting an array of strings (`char *`) works the same way: each element is a `char *`, so the comparator receives `char **`s:

```
int compare_strs(const void *a, const void *b) {
  const char * const *sa = a;
  const char * const *sb = b;
  return strcmp(*sa, *sb);
}
```

13.2.2 A generic BST dictionary

```
struct dictionary {
  struct bstnode *root;
  int (*key_compare)(const void *, const void *);
};

struct dictionary *dict_create(int (*cmp)(const void *, const void *)) {
  struct dictionary *d = malloc(sizeof(struct dictionary));
  d->root = NULL;
  d->key_compare = cmp;
  return d;
}

void *dict_lookup(const void *key, const struct dictionary *d) {
  const struct bstnode *node = d->root;
  while (node) {
    int result = d->key_compare(key, node->key);
    if (result == 0) return node->val;
    node = (result < 0) ? node->left : node->right;
  }
  return NULL;
}
```

This is a limited form of what Racket gets for free with higher-order functions, and what C++ achieves with templates (CS 246).

13.3 `typedef`

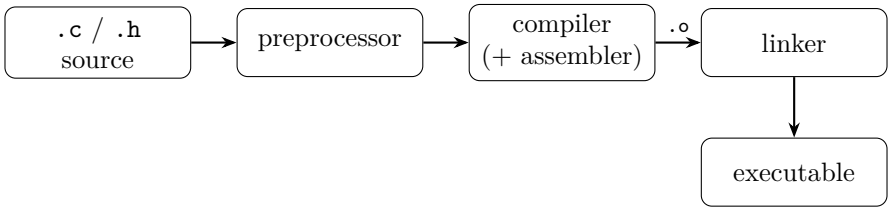
`typedef` creates an alias for a type. CS 136 style mostly avoided it (so that pointers and structures stay visible), but it is common in real code, especially for function pointer types:

```
typedef int (*CompareFn)(const void *, const void *);
typedef struct dictionary *Dictionary;   // hides the pointer: use carefully

Dictionary dict_create(CompareFn cmp);
```

13.4 The Compilation Pipeline

Seashell hides how a C program becomes an executable. Running `gcc` on several files does four steps:



1. **Preprocessing:** handles directives starting with `#`: `#include` pastes in header files, `#define` does textual macro substitution, `#ifdef` includes code conditionally. The result is a single "translation unit" per `.c` file.
2. **Compiling:** each translation unit is checked (types, syntax) and translated to assembly language and then (by the *assembler*) to machine code in an *object file* (`.o`). References to functions in other modules are left unresolved. This is why a `.c` file only needs the *declarations* from headers.
3. **Linking:** the linker combines all object files and libraries (e.g. the C standard library for `printf`), resolves every reference to its definition, and produces the executable. "Undefined reference" errors and duplicate-definition errors come from this step.

```
gcc -std=c99 -Wall -c stack.c        # produces stack.o
gcc -std=c99 -Wall -c main.c         # produces main.o
gcc stack.o main.o -o prog           # link
./prog < test1.in
```

Separate compilation means that when one module changes, only that module needs to be recompiled before relinking. Tools like `make` automate this (CS 246).

13.4.1 Include guards

If a header is included twice in one translation unit (e.g. directly and through another header), structure definitions would be duplicated. Headers are therefore protected:

```
#ifndef STACK_H
#define STACK_H
// ... declarations ...
#endif
```

13.4.2 Command-line arguments

Outside Seashell, `main` can receive arguments typed on the command line:

```
int main(int argc, char *argv[]) {
  // argc: number of arguments (including the program name)
  // argv: array of strings; argv[0] is the program name
  for (int i = 1; i < argc; ++i) {
    printf("argument %d: %s\n", i, argv[i]);
```

```
  }
}
```

13.5 C vs. Racket vs. Other Languages

	Racket	C	C++ / Java / Python
Paradigm	functional (mostly)	imperative	imperative + object-oriented
Typing	dynamic	static (weak: casts, `void *`)	static (C++, Java), dynamic (Python)
Memory	garbage collected	manual `malloc`/`free`	RAII/destructors (C++), GC (Java, Python)
Execution	interpreted / JIT	compiled to machine code	compiled (C++), bytecode VM (Java), interpreted (Python)
Generic code	automatic	`void *`, function pointers	templates / generics / duck typing
Safety	errors reported	undefined behaviour	exceptions, bounds checks (Java, Python)

- **Garbage collection:** Racket, Java and Python automatically free memory that is no longer reachable, so leaks and dangling pointers (mostly) cannot happen, at some cost in run-time overhead and control. C gives full control and full responsibility.
- **Object-oriented programming** (CS 246) bundles data with the functions that operate on it (*classes*), with access control (`private`) enforcing information hiding — a language-level version of our opaque-structure ADTs. Constructors and destructors replace `_create` and `_destroy`.
- **Undefined behaviour** is C's trade-off for speed: no bounds checks, no overflow checks. Many security vulnerabilities (buffer overflows) come from it.
- Despite its age, C remains essential for operating systems, embedded systems, and performance-critical libraries, and its memory model underlies how every other language is implemented.

13.6 Where the Ideas Go Next

- **CS 246** (Object-Oriented Software Development): C++, classes, RAII, templates, design patterns, `make`.
- **CS 241** (Foundations of Sequential Programs): how the compiler, assembler and linker actually work; MIPS assembly; how the stack and heap are managed.
- **CS 240** (Data Structures and Data Management): balanced trees, hash tables, priority queues, sorting lower bounds, and more rigorous analysis.

- **CS 251** (Computer Organization and Design): how memory, bits and machine code are realised in hardware.
- **CS 350** (Operating Systems): processes, virtual memory — the segmentation faults we saw come from here.

13.7 Summary

- `void *` allows generic ADTs; the interface must define ownership of items.
- Function pointers (comparators like `qsort`'s) make algorithms generic.
- Building a program: preprocess → compile/assemble to object files → link. Headers hold declarations; include guards prevent double inclusion.
- C trades safety for control and speed; other languages add garbage collection, objects and generics on top of the same memory model.